



FACULTÉ DES SCIENCES INFORMATIQUES
Filière : Intelligence Artificielle

**Navigateur IA : Conception et développement d'une
plateforme intelligente modulaire embarquée dans un
casque de réalité assistée pour l'assistance contextuelle**

*Travail de fin d'études présenté et défendu en vue de l'obtention
du grade de licencié en sciences informatiques.*

Présenté par : Mulongoy Lubabila Adriel

Dirigé par : Prof. Antoin Bagula

Codirigé par : Ass. Landry Mbale

Juillet 2026



FACULTÉ DES SCIENCES INFORMATIQUES
Filière : Intelligence Artificielle

**Navigateur IA : Conception et développement d'une
plateforme intelligente modulaire embarquée dans un
casque de réalité assistée pour l'assistance contextuelle**

*Travail de fin d'études présenté et défendu en vue de l'obtention
du grade de licencié en sciences informatiques.*

Présenté par : Mulongoy Lubabila Adriel

Dirigé par : Prof. Antoin Bagula

Codirigé par : Ass. Landry Mbale

Juillet 2026

DÉCLARATION D'ORIGINALITÉ

Je soussigné, **Mulongoy Lubabila Adriel**, étudiant en Licence à la Faculté des Sciences Informatiques, filière Intelligence Artificielle, déclare sur l'honneur que le présent mémoire, intitulé « *Navigateur IA : Conception et développement d'une plateforme intelligente modulaire embarquée dans un casque de réalité assistée pour l'assistance contextuelle* », est le fruit de mon travail personnel, réalisé sous la direction du Prof. Antoin Bagula et la codirection de l'Ass. Landry Mbale.

Je certifie que :

- ce travail n'a jamais été présenté, en tout ou en partie, pour l'obtention d'un autre grade académique, dans cette institution ou ailleurs ;
- toutes les sources d'information utilisées (articles scientifiques, ouvrages, documentation technique, ressources en ligne) sont citées et référencées conformément au style bibliographique IEEE ;
- les mesures et résultats expérimentaux présentés proviennent d'exécutions réelles sur le prototype décrit, sans falsification ni extrapolation non signalée ;
- je suis conscient que toute forme de plagiat ou de fraude académique constitue une violation grave du règlement de la Faculté et expose son auteur aux sanctions prévues.

Fait à Kinshasa, le 4 juillet 2026

Signature de l'étudiant

.....

Mulongoy Lubabila Adriel

REMERCIEMENTS

Je tiens d'abord à exprimer ma profonde gratitude à mon directeur, le Professeur **Antoin Bagula**, pour son encadrement, ses orientations scientifiques et la confiance qu'il m'a accordée tout au long de ce travail.

Mes remerciements vont également à mon codirecteur, l'Assistant **Landry Mbale**, pour sa disponibilité, ses relectures attentives et ses conseils techniques précieux.

Je remercie le corps enseignant de la Faculté des Sciences Informatiques pour la formation reçue durant ce cycle de Licence, ainsi que [camarades, testeurs, laboratoire, etc.] qui ont participé aux sessions d'évaluation du prototype.

Enfin, je remercie ma famille et mes proches, [], pour leur soutien indéfectible.

RÉSUMÉ

L'intelligence artificielle embarquée et les capteurs miniaturisés rendent aujourd'hui possibles, sur du matériel grand public, des fonctions de perception autrefois réservées à des stations de calcul dédiées. Les dispositifs d'assistance portés existants restent cependant monolithiques, dédiés à une tâche unique et souvent dépendants d'une connexion réseau, ce qui limite leur fiabilité pour des usages critiques tels que l'assistance aux personnes malvoyantes ou la télé-maintenance industrielle.

Ce mémoire pose la question scientifique suivante : comment concevoir une plateforme d'assistance portée, exécutée entièrement sur du matériel embarqué à ressources limitées, qui maintienne une latence compatible avec la sécurité de l'utilisateur tout en restant modulaire, reconfigurable et indépendante de toute connexion réseau ?

La réponse proposée est Nova, une plateforme cyber-physique organisée en quatre couches (Physique, Middleware, Traitement Edge AI, Application) autour d'un bus de messages publish/subscribe (MQTT). Un Raspberry Pi 4B orchestre des modules logiciels indépendants chargés dynamiquement sous forme de greffons, un Arduino Uno assure l'acquisition temps réel des capteurs (ultrason, environnement) et le retour haptique, et un ESP32-CAM fournit un flux vidéo déporté. La perception combine MediaPipe (commande gestuelle), la reconnaissance vocale hors ligne (Vosk) et la lecture de texte (Tesseract), tandis qu'un système de profils adapte les combinaisons de modules à chaque contexte d'usage.

Les mesures réalisées sur Raspberry Pi 4B, consolidées sur trois exécutions, établissent une validation embarquée de l'architecture : latence médiane du bus MQTT de 1,82 ms sans perte de paquets observée, fonctionnement intégralement hors ligne, occupation mémoire moyenne de 27,3 %, débit vision moyen de 2,0 FPS, charge CPU moyenne de 93,7 % et température maximale de 58,4 °C. L'autonomie mesurée atteint 7 h 30 sur batterie Li-Po 7,4 V / 4 400 mAh (32,6 Wh) pour une cible de 8 h. La reconnaissance vocale atteint un taux d'erreur de mots de 14,3 % (évaluation en direct sur 10 locuteurs) et l'OCR un taux de reconnaissance de 87 % sur 150 images de texte en conditions favorables. La latence bout-en-bout de la chaîne de navigation n'a pas encore été instrumentée et constitue le travail prioritaire à venir.

Mots-clés : intelligence artificielle embarquée ; assistance aux personnes malvoyantes ; architecture modulaire ; MQTT ; Raspberry Pi ; vision par ordinateur ; MediaPipe ; systèmes cyber-physiques ; Edge AI ; temps réel.

ABSTRACT

Embedded artificial intelligence and miniaturized sensors now make perception capabilities available on consumer-grade hardware that previously required dedicated computing stations. Existing wearable assistive devices, however, remain monolithic, single-purpose, and often network-dependent, which limits their reliability for critical use cases such as assistance for visually impaired people or industrial tele-maintenance.

This thesis addresses the following scientific question : how can a wearable assistance platform be designed to run entirely on resource-constrained embedded hardware while maintaining user-safety-compatible latency and remaining modular, reconfigurable, and fully independent of any network connection ?

The proposed answer is Nova, a cyber-physical platform organized into four layers (Physical, Middleware, Edge AI Processing, Application) around a publish/subscribe message bus (MQTT). A Raspberry Pi 4B orchestrates independent software modules dynamically loaded as plugins, an Arduino Uno handles real-time sensor acquisition (ultrasonic, environmental) and haptic feedback, and an ESP32-CAM provides a remote video stream. Perception combines MediaPipe (gesture control), offline speech recognition (Vosk), and text reading (Tesseract), while a profile system adapts module combinations to each usage context.

Measurements performed on the Raspberry Pi 4B and consolidated over three runs provide an embedded validation of the architecture : a median MQTT bus latency of 1.82 ms with no observed packet loss, fully offline operation, average memory usage of 27.3 %, an average vision throughput of 2.0 FPS, average CPU usage of 93.7 %, and a maximum temperature of 58.4 °C. Measured battery life reaches 7 h 30 on a 7.4 V / 4,400 mAh Li-Po battery (32.6 Wh), against an 8 h target. Speech recognition achieves a 14.3 % word error rate (live evaluation with 10 speakers) and OCR an 87 % recognition rate on 150 text images under favorable conditions. End-to-end latency of the navigation chain has not yet been instrumented and is the primary future work.

Keywords : embedded artificial intelligence ; assistance for visually impaired people ; modular architecture ; MQTT ; Raspberry Pi ; computer vision ; MediaPipe ; cyber-physical systems ; Edge AI ; real-time.

TABLE DES MATIÈRES

Déclaration d'originalité	iii
Remerciements	iv
Résumé	v
Abstract	vi
Liste des acronymes	xii
1 INTRODUCTION GÉNÉRALE	1
1.1 Contexte et motivation	1
1.2 Problématique générale	1
1.3 Objectifs du travail	2
1.3.1 Objectif principal	2
1.3.2 Objectifs spécifiques	2
1.4 Contributions du mémoire	2
1.4.1 Contributions scientifiques	2
1.4.2 Contributions techniques	3
1.5 Limites et périmètre du travail	3
2 ÉTAT DE L'ART ET REVUE DE LITTÉRATURE	4
2.1 Introduction	4
2.2 Travaux existants	4
2.2.1 Systèmes assistifs basés sur l'IA et la vision	4
2.2.2 Architectures matérielles et fusion de capteurs	5
2.2.3 Synthèse des approches actuelles	6
2.3 Analyse comparative	7
2.3.1 Performance et latence	7
2.3.2 Complexité architecturale	7
2.3.3 Robustesse et fiabilité	7
2.3.4 Synthèse comparative	7
2.4 Limites identifiées	7
2.4.1 Insuffisances méthodologiques	7
2.4.2 Faiblesses expérimentales	8
2.4.3 Contraintes techniques	8
2.4.4 Conclusion et nécessité d'une nouvelle approche	9
3 MÉTHODOLOGIE	10
3.1 Architecture proposée	10
3.1.1 Description globale du système	10
3.1.2 Flux de données	10
3.1.3 Justification des choix architecturaux	11
3.1.4 Analyse fonctionnelle (UML)	12
3.1.5 Architecture et Modélisation Logicielle	12
3.2 Modélisation	15
3.2.1 Formulation mathématique	15
3.2.2 Hypothèses de travail	15

3.2.3	Variables du système	15
3.3	Algorithmes	16
3.3.1	Classification des gestes (MediaPipe)	16
3.3.2	Détection d'obstacles	18
3.3.3	Workflow dynamique du système : cycle de vie d'un événement critique . .	18
3.3.4	Chargement dynamique des modules	19
4	CHOIX TECHNIQUES	20
4.1	Technologies utilisées	20
4.2	Justification des choix de conception	20
4.3	Architecture système en quatre couches	22
4.4	Contraintes prises en compte	23
5	IMPLÉMENTATION	24
5.1	Environnement expérimental	24
5.1.1	Configuration matérielle	24
5.1.2	Logiciels utilisés et versions	36
5.2	Développement	36
5.2.1	Organisation du code	37
5.2.2	Bibliothèques utilisées	37
5.2.3	Structure d'un module plugin	38
5.3	Jeux de données	38
5.3.1	Source des données	38
5.3.2	Prétraitement	38
5.3.3	Description statistique	38
6	RÉSULTATS ET ANALYSE	40
6.1	Métriques et protocole d'évaluation	40
6.1.1	Plateforme et protocole	40
6.1.2	Métriques retenues	40
6.2	Résultats expérimentaux	40
6.2.1	Latence du bus de messages MQTT	40
6.2.2	Débit du pipeline vision	41
6.2.3	Charge système	41
6.2.4	Reconnaissance vocale et OCR	41
6.2.5	Autonomie énergétique	42
6.3	Analyse critique	42
6.3.1	Le bus MQTT valide le choix architectural	42
6.3.2	Le facteur limitant est le calcul, non la chaleur	42
6.3.3	Interprétation du débit vision	42
6.3.4	Limite de validation : la latence bout-en-bout	42
6.3.5	Synthèse des résultats par rapport aux objectifs	43
7	DISCUSSION	44
7.1	Interprétation globale	44
7.1.1	Ce que disent les résultats	44
7.1.2	Le vrai enseignement : où se situe la contrainte	44
7.1.3	Une contribution architecturale, non algorithmique	44
7.1.4	Retour sur les quatre tensions de la problématique	45
7.2	Forces du travail	45
7.2.1	Points positifs	45
7.2.2	Robustesse	45

7.2.3	Innovation	46
8	LIMITES	47
8.1	Hypothèses Restrictives	47
8.1.1	Hypothèse de luminosité suffisante	47
8.1.2	Hypothèse de main visible dans le champ de la caméra	47
8.1.3	Hypothèse d'obstacles physiquement solides	47
8.1.4	Hypothèse d'un environnement acoustique calme pour le STT	47
8.2	Contraintes Techniques	48
8.2.1	Contrainte CPU : deux modules simultanés maximum	48
8.2.2	Contrainte mémoire RAM	48
8.2.3	Autonomie batterie proche de la cible	48
8.3	Limites de Généralisation	49
8.3.1	Validation en laboratoire uniquement	49
8.3.2	Absence de validation avec des utilisateurs malvoyants	49
8.3.3	Architecture non testée à grande échelle	49
8.3.4	Dépendance aux versions spécifiques des bibliothèques	50
9	CONCLUSION ET PERSPECTIVES	51
9.1	Synthèse des contributions	51
9.1.1	Le problème traité	51
9.1.2	L'approche proposée	51
9.1.3	Les résultats principaux	51
9.2	Apports scientifiques	51
9.3	Perspectives	52
9.3.1	Améliorations possibles	52
9.3.2	Extensions futures	52
9.3.3	Applications potentielles	52
9.3.4	Recherches complémentaires	53
A	GUIDE INSTALLATION SYSTEME NOVA	54
A.1	Prérequis système	54
A.2	Structure du projet	54
A.3	Lancement du système	55
A.4	Format des messages MQTT	55
	Bibliographie	56

TABLE DES FIGURES

Figure 1.1	Diagramme de contexte statique de la plateforme Nova	3
Figure 2.1	Pipeline de traitement du flux de vision par ordinateur (YOLOv8n)	4
Figure 3.1	Architecture matérielle distribuée de la plateforme Nova	11
Figure 3.2	Architecture globale de la plateforme Nova	12
Figure 3.3	Diagramme de cas d'utilisation globale du système Nova	13
Figure 3.4	Diagramme de classes de l'application centrale de Nova	14
Figure 3.5	Architecture fonctionnelle Nova : flux dynamique	19
Figure 4.1	Synergie et déclenchement conditionnel des modèles d'IA embarqués	21
Figure 5.1	Masque de protection servant de châssis porteur au système Nova	24
Figure 5.2	Raspberry Pi 4 Model B unité centrale du système Nova	25
Figure 5.3	Arduino Uno Rev3 hub d'acquisition des capteurs bas niveau	26
Figure 5.4	Module ESP32-CAM avec capteur OV2640	27
Figure 5.5	Capteur ultrasonique HC-SR04	28
Figure 5.6	Capteur DHT11 pour température et humidité	28
Figure 5.7	Capteur de gaz MQ-3	29
Figure 5.8	Récepteur infrarouge pour détection de signaux IR	30
Figure 5.9	Récepteur infrarouge de proximité pour la couverture des angles morts latéraux	31
Figure 5.10	Joystick analogique 2 axes	32
Figure 5.11	Clavier matriciel 4×4 16 touches	32
Figure 5.12	Potentiomètre rotatif 10 kΩ	33
Figure 5.13	Microphone intégré aux écouteurs du casque	33
Figure 5.14	Moteur à vibration type 130 : retour haptique	33
Figure 5.15	Haut-parleur 8 Ω / 0,5 W pour la synthèse vocale	34
Figure 5.16	Buzzer actif : alertes sonores immédiates	34
Figure 5.17	LEDs de statut RGB : indicateurs visuels du système	35
Figure 5.18	Convertisseur de niveau logique bidirectionnel	36

LISTE DES TABLEAUX

Table 1	Comparatif des travaux existants et positionnement de la solution proposée	6
Table 2	Comparaison critique des approches assistives	8
Table 3	Variables principales du modèle système Nova	15
Table 4	Bibliothèque des quatre gestes implémentés dans <code>hand_control/main.py</code>	16
Table 5	Caractéristiques physiques du masque de protection	25
Table 6	Spécifications techniques du Raspberry Pi 4B	25
Table 7	Spécifications techniques de l'Arduino Uno Rev3	26
Table 8	Spécifications techniques de l'ESP32-CAM	27
Table 9	Spécifications techniques du capteur HC-SR04	28
Table 10	Spécifications techniques du capteur DHT11	29
Table 11	Spécifications techniques du MQ-3	30
Table 12	Spécifications techniques du capteur IR de proximité	30
Table 13	Spécifications techniques du capteur MPU-6050	31
Table 14	Spécifications du vibreur type 130	34
Table 15	Stack logicielle complète du prototype Nova V3.0	36
Table 16	Caractéristiques des jeux de données d'évaluation (10 testeurs , 3 sessions chacun)	39
Table 17	Latence du bus MQTT local mesurée sur Raspberry Pi 4B	40
Table 18	Débit du pipeline vision observé par benchmark SHOS (60 s)	41
Table 19	Charge système mesurée sur Raspberry Pi 4B	41
Table 20	Résultats mesurés au regard des objectifs	43
Table 21	Consommation mémoire des composants du système Nova	48

LISTE DES ACRONYMES

API	Application Programming Interface
BGR/RGB	Blue-Green-Red / Red-Green-Blue (formats d'image)
CPU	Central Processing Unit
DSR	Design Science Research
E2E	End-to-End (bout-en-bout)
FPS	Frames Per Second (images par seconde)
GPIO	General Purpose Input/Output
HTTP	HyperText Transfer Protocol
I2C	Inter-Integrated Circuit
IA	Intelligence Artificielle
IMU	Inertial Measurement Unit (centrale inertielle)
IoT	Internet of Things (Internet des objets)
IR	Infrarouge
LDR	Light Dependent Resistor (photorésistance)
LLM	Large Language Model
MJPEG	Motion JPEG
MQTT	Message Queuing Telemetry Transport
OCR	Optical Character Recognition
OMS	Organisation Mondiale de la Santé
P95/P99	95 ^e / 99 ^e centile
PWM	Pulse Width Modulation
QoS	Quality of Service
RAM	Random Access Memory
SoC	System on Chip
STT/TTS	Speech-To-Text / Text-To-Speech
TinyML	Tiny Machine Learning
UART	Universal Asynchronous Receiver-Transmitter
WER	Word Error Rate (taux d'erreur de mots)
YOLO	You Only Look Once

INTRODUCTION GÉNÉRALE

L'intelligence artificielle embarquée et les capteurs miniaturisés ont récemment rendu accessibles, sur du matériel grand public, des fonctions de perception qui exigeaient hier des stations de calcul dédiées. Cette bascule ouvre une voie concrète pour les dispositifs d'assistance portés sur soi. Ce mémoire présente Nova, une plateforme modulaire embarquée dans un casque de réalité assistée, dont le nom résume les quatre piliers fonctionnels : Navigation, Orchestration, Vision et Assistance.

1.1 CONTEXTE ET MOTIVATION

La nécessité de concevoir des systèmes d'assistance portés intelligents s'impose aujourd'hui dans plusieurs domaines critiques. En premier lieu, la santé publique est concernée : l'Organisation mondiale de la santé estime à au moins 2,2 milliards le nombre de personnes souffrant d'une déficience visuelle [1]. Les outils classiques d'aide à la navigation (cane blanche, chien-guide) n'offrent qu'une perception limitée, ignorant les obstacles en hauteur et n'apportant aucune analyse sémantique de la scène visuelle. En second lieu, le secteur industriel exprime un besoin similaire en matière de télé-assistance et de maintenance cognitive, où l'opérateur doit interagir les mains libres tout en exploitant un flux vidéo distant.

Les solutions technologiques actuelles développées pour ces deux cas d'usage partagent une limitation majeure : leur rigidité architecturale. Conçues comme des systèmes monolithiques dédiés à une tâche unique, elles ne permettent aucune reconfiguration. De ce constat naît l'objet de ce mémoire : le développement de la plateforme Nova, une architecture cyber-physique modulaire capable d'orchestrer et de combiner des briques logicielles (Edge AI) et matérielles interchangeables en fonction des besoins spécifiques de l'utilisateur et de son environnement.

1.2 PROBLÉMATIQUE GÉNÉRALE

Concevoir un tel système se heurte à un verrou que la simple amélioration d'un algorithme ne lève pas. La question scientifique centrale de ce mémoire est la suivante :

Comment concevoir une plateforme d'assistance portée, exécutée entièrement sur du matériel embarqué à ressources limitées, qui maintienne une latence compatible avec la sécurité de l'utilisateur tout en restant modulaire, reconfigurable et indépendante de toute connexion réseau ?

Cette question se décompose en quatre tensions concrètes :

1. **Ressources contre temps réel.** Les modèles d'IA les plus riches sémantiquement sont aussi les plus lourds. Sur un nano-ordinateur, ils saturent le processeur et introduisent des délais incompatibles avec une alerte d'obstacle utile. Comment obtenir une réponse sous le seuil de sécurité visé (de l'ordre de 200 ms) sans renoncer au sens ?
2. **Monolithe contre modularité.** Empiler les fonctions dans un programme unique rend le système rigide et fragile : une panne arrête tout. Comment isoler les fonctions de sorte que l'arrêt de l'une n'interrompe pas les autres ?
3. **Usage unique contre multi-contexte.** Un même casque devrait servir aussi bien l'aide aux personnes malvoyantes que la maintenance ou la formation. Comment adapter le comportement du dispositif au contexte et aux préférences de l'utilisateur sans réécrire le logiciel ?

4. **Dépendance réseau contre autonomie.** Une personne malvoyante peut se trouver dans un sous-sol, un tunnel ou une zone sans couverture. Comment garantir que toute la chaîne de traitement fonctionne hors ligne ?

1.3 OBJECTIFS DU TRAVAIL

1.3.1 *Objectif principal*

Concevoir, mettre en œuvre et évaluer une plateforme matérielle et logicielle modulaire, embarquée sur un casque, offrant plusieurs modes d’usage orchestrés depuis une interface de configuration en réseau local, sous contrainte de temps réel et de fonctionnement hors ligne.

1.3.2 *Objectifs spécifiques*

- **Plateforme matérielle.** Intégrer un Raspberry Pi 4B (unité centrale), un Arduino Uno (acquisition des capteurs temps réel), un ESP32-CAM (flux vidéo déporté) et l’ensemble des capteurs et actionneurs (ultrason, IMU, environnement, retour haptique et sonore).
- **Orchestration logicielle.** Développer un orchestrateur fondé sur un bus de messages léger (MQTT) reliant des modules logiciels indépendants, chargés dynamiquement sous forme de greffons (*plugins*).
- **Modules fonctionnels.** Implémenter les fonctions clés : détection d’obstacles (ultrason et vision), reconnaissance et synthèse vocale hors ligne, lecture de texte, retour haptique et streaming vidéo pour la télé-assistance.
- **Commande multi-modale.** Offrir deux modalités d’entrée complémentaires, le geste de la main et le joystick physique, sélectionnées selon le contexte d’usage.
- **Profils utilisateurs.** Permettre la configuration de combinaisons de modules adaptées à chaque cas d’usage et persistantes entre les sessions.
- **Évaluation.** Établir un protocole de mesure reproductible portant sur la latence, le débit du pipeline de vision et la charge système, instrumenté par un banc de *benchmark* dédié.

1.4 CONTRIBUTIONS DU MÉMOIRE

Les contributions revendiquées correspondent à ce que le prototype réalise et mesure effectivement. Elles sont les suivantes.

1.4.1 *Contributions scientifiques*

- **Un modèle d’architecture modulaire orchestrée.** Une organisation logicielle dans laquelle des modules interchangeables communiquent par un bus de messages publish/subscribe et sont découverts au démarrage sans modification du code central. L’ajout d’une fonction se fait par dépôt d’un dossier de greffon.
- **Une chaîne d’assistance entièrement hors ligne.** La démonstration qu’une perception multimodale (gestes, parole, lecture de texte) peut s’exécuter localement sur un nano-ordinateur, sans aucune dépendance à un service distant.
- **Un cadre d’évaluation reproductible.** Un ensemble de métriques et un protocole de mesure (latence du bus, débit du pipeline vision, charge processeur et mémoire) applicables à ce type de système.

1.4.2 Contributions techniques

- **Un prototype intégré et documenté.** Un assemblage matériel fonctionnel (Pi 4B, Arduino, ESP32-CAM, capteurs et actionneurs) piloté par une pile logicielle Python/Flask et MQTT.
- **Une commande multi-modale contextuelle.** Une interface d'entrée combinant le contrôle gestuel par vision et un joystick physique, chacun activé selon les conditions d'usage (mains occupées, faible luminosité).
- **Un système de profils configurables.** Un questionnaire associant à chaque cas d'usage une combinaison de modules, modifiable depuis une interface web accessible en réseau local.

Afin de synthétiser visuellement le périmètre d'action de notre solution et d'en clarifier les frontières avant d'aborder ses spécificités internes, la Figure 1.1 présente la cartographie globale de la plateforme. Le dispositif Nova y est appréhendé comme une entité centrale interconnectée : il collecte de manière passive les informations de son environnement (flux vidéo et mesures de distance), traite ces données localement grâce à ses algorithmes embarqués, puis restitue une assistance contextuelle à l'utilisateur malvoyant sous forme de retours haptiques et d'alertes vocales. Par ailleurs, l'interface web Flask assure l'ouverture du système vers un écosystème connecté pour la supervision.

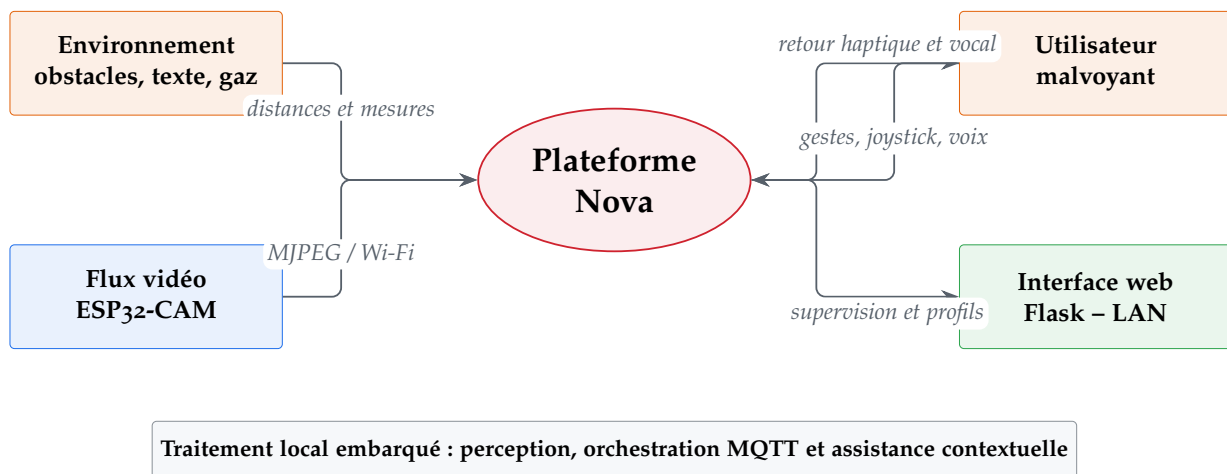


FIGURE 1.1 – Diagramme de contexte statique de la plateforme Nova

1.5 LIMITES ET PÉRIMÈTRE DU TRAVAIL

Ce mémoire se positionne comme une preuve de concept académique : il vise à établir la faisabilité technique et scientifique d'une architecture modulaire multi-usage pour casque d'assistance, non un produit fini. Les bornes suivantes sont posées d'emblée et discutées en détail au chapitre 8.

- Le prototype ne vise ni la commercialisation ni l'industrialisation.
- Les traitements les plus gourmands (grands modèles de langage, reconstruction volumétrique de l'environnement) sont envisagés comme options déportées (smartphone ou serveur) plutôt qu'embarqués.
- La validation utilisateur reste limitée en échelle et en durée, pour des raisons éthiques et de calendrier.
- L'intégration de capteurs coûteux (LiDAR, caméra événementielle) reste optionnelle et relève des perspectives.

ÉTAT DE L'ART ET REVUE DE LITTÉRATURE

2.1 INTRODUCTION

Trois évolutions parallèles rendent aujourd'hui crédible un dispositif d'assistance porté et autonome : l'IA embarquée légère (TinyML), des plateformes matérielles à faible coût (Raspberry Pi, ESP32, Arduino) et des protocoles de communication adaptés aux objets connectés. Pourtant, la littérature récente montre que ces briques restent le plus souvent juxtaposées plutôt qu'intégrées : peu de travaux proposent une architecture à la fois modulaire, extensible et capable de fonctionner sans connexion au cloud.

Ce chapitre examine les travaux pertinents pour le projet Nova selon deux axes : les systèmes assistifs fondés sur l'IA et la vision, puis les architectures matérielles et les stratégies de fusion de capteurs. L'analyse comparative qui suit met en évidence les limites structurelles qui motivent la contribution de ce mémoire : une plateforme modulaire, hors ligne et reconfigurable à faible latence.

2.2 TRAVAUX EXISTANTS

2.2.1 Systèmes assistifs basés sur l'IA et la vision

Détection d'objets embarquée (YOLO)

L'approche la plus répandue dans la littérature couple des lunettes connectées à un smartphone exécutant un détecteur d'objets. Cho et al. [1] proposent un système de lunettes XR pour l'aide à la marche en extérieur. Déployé sur un smartphone Android relié à des lunettes Xreal Light, il s'appuie sur un modèle YOLOv8n entraîné sur cinq classes d'éléments urbains (trottoirs, passages piétons, arrêts de bus, sorties de métro, obstacles divers), pour un temps de traitement moyen de 583 ms et une consommation mémoire élevée.

L'architecture divise le champ visuel en neuf zones et calcule pour chacune un niveau de danger. Ce travail est l'un des rares à être évalué en conditions extérieures réelles, ce qui renforce sa validité par rapport aux études de laboratoire. Sa limite est cependant rédhibitoire pour la sécurité : une latence bout-en-bout de 583 ms crée, en déplacement, un décalage entre la position réelle de l'utilisateur et les alertes affichées, ce qui compromet l'évitement d'obstacles en temps réel. L'architecture est par ailleurs monolithique (un modèle, un smartphone, un affichage), sans possibilité d'extension modulaire.

Pour comprendre la manière dont l'appareil perçoit l'environnement, il est nécessaire de détailler le cheminement des données visuelles. La Figure 2.1 schématise les différentes étapes du pipeline de vision par ordinateur implémenté au sein de Nova. Ce traitement local garantit une extraction rapide des descripteurs sémantiques indispensables à la prise de décision.

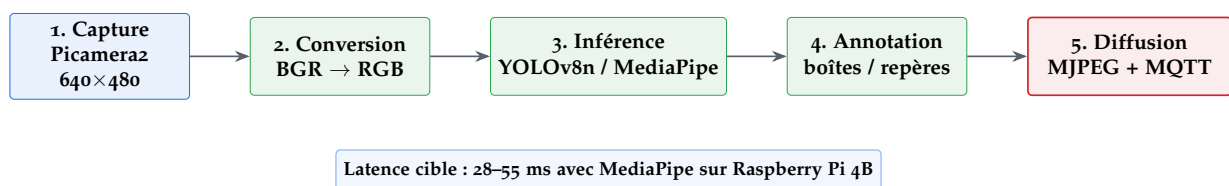


FIGURE 2.1 – Pipeline de traitement du flux de vision par ordinateur (YOLOv8n)

Systèmes combinant LLM et retour haptique

Tokmurziyev et al. [2] suivent une voie très différente avec LLM-Glasses, un système de navigation portable associant retour haptique, détection d'objets YOLO-World et raisonnement par GPT-4o, qui délivre une guidance tactile directionnelle au moyen d'actionneurs montés sur les tempes.

Trois études utilisateurs en documentent les performances : une reconnaissance haptique atteignant 81,3 % sur treize motifs distincts, une étude de navigation en espace ouvert, et une évaluation du raisonnement par LLM donnant 91,8 % de précision en scénario ouvert, 84,6 % avec obstacles statiques et 81,5 % avec obstacles dynamiques.

L'apport est double : le retour haptique encode une information directionnelle avec un taux d'apprentissage acceptable, et le raisonnement sémantique par LLM se révèle pertinent pour la navigation. La limite est tout aussi nette : la dépendance à GPT-4o impose une latence cloud de 2 à 5 secondes, incompatible avec la détection d'obstacles dynamiques en temps réel. Le coût de prototypage reste modeste (environ 150 dollars, ESP32-CAM et pièces imprimées en 3D), mais le système est inutilisable hors connexion.

Modalités d'interaction et interfaces de commande

Le contrôle par geste de la main est un canal d'interaction sans contact fréquemment étudié pour les casques d'assistance. Deux familles se distinguent : les modèles généralistes (YOLO-Pose, CNN) et les *frameworks* spécialisés comme MediaPipe. Andriyanov et Mikhailova [3] montrent qu'en combinant MediaPipe pour l'extraction rapide des points-clés et YOLO-Pose pour l'affinage contextuel, on atteint une précision mAP supérieure à 0,80, y compris sous des angles de prise de vue extrêmes. Sur Raspberry Pi 4B, MediaPipe Hands [4] détecte 21 repères 3D avec une latence d'inférence de 28 à 55 ms, contre 120 à 200 ms pour un modèle YOLOv8n PyTorch complet, et la littérature rapporte une empreinte mémoire nettement inférieure.

Le geste a toutefois des limites réelles dans le contexte d'un usage urbain par une personne malvoyante. Sans retour visuel, l'utilisateur peine à maintenir sa main dans le champ de la caméra. Effectuer des gestes en l'air en public fatigue le bras et manque de discrétion. La détection chute enfin sous une certaine luminosité.

Ces contraintes ne disqualifient pas le geste : elles montrent qu'*aucune modalité d'entrée unique ne convient à tous les contextes*. Or les systèmes assistifs de la littérature s'engagent presque toujours sur un seul canal de commande. C'est ce constat qui justifie l'un des choix de Nova : proposer deux modalités complémentaires, le geste (MediaPipe) lorsque l'utilisateur ne peut pas toucher l'appareil, et un joystick physique lorsque le geste est inopérant (faible luminosité, besoin d'un retour tactile immédiat). Ce choix est détaillé au chapitre 4.

2.2.2 Architectures matérielles et fusion de capteurs

Vision stéréoscopique et estimation de profondeur

La vision stéréoscopique estime la distance métrique des obstacles sans capteur actif. Son atout est la détection d'obstacles aériens (branches, enseignes basses) que ni la canne blanche ni l'ultrason ne signalent. Deux limites en freinent l'usage sur un système embarqué contraint : la dépendance à la texture des surfaces (les surfaces lisses empêchent la mise en correspondance des points) et la sensibilité à la lumière (contre-jour, nuit). Le calcul d'une carte de disparité dense réclame en outre entre 200 et 800 ms sur un Raspberry Pi 4B, ce qui exclut la navigation en temps réel.

Systèmes hybrides multi-capteurs

Les approches les plus robustes combinent vision et capteurs de distance physiques. Fusionner données ultrasoniques et visuelles permet de hiérarchiser les alertes selon la distance mesurée et d'améliorer la fiabilité face aux obstacles transparents ou aux surfaces sombres mal détectées par la caméra. Abid et al. [5] montrent qu'une architecture distribuée, avec l'ESP32 comme nœud de capture et MQTT comme couche de communication, atteint une latence de message inférieure à 5 ms en réseau local, tout en faisant coexister des capteurs hétérogènes sans couplage logiciel direct.

Protocoles de communication pour systèmes embarqués

MQTT (*Message Queuing Telemetry Transport*) est le protocole de communication IoT dominant, avec une adoption de 56 % d'après l'enquête Eclipse Foundation 2024 [6]. Il repose sur un modèle publish/subscribe : les producteurs publient sur des *topics*, et un *broker* central redistribue les messages aux abonnés sans que les parties se connaissent. Cette organisation offre trois propriétés utiles à un système d'assistance.

1. **Découplage fort.** Un module peut tomber en panne sans affecter les autres ; le broker gère la déconnexion silencieusement.
2. **Surcharge minimale.** L'en-tête fixe de MQTT est de 2 octets au minimum, contre 26 à 80 octets pour HTTP/1.1. Pour des messages de capteurs de moins de 256 octets, l'écart pèse sur la latence, notamment sur des liaisons UART à 9600 bauds.
3. **Qualité de service configurable.** Les niveaux QoS₀ (au plus une fois), QoS₁ (au moins une fois) et QoS₂ (exactement une fois) adaptent la garantie de livraison à la criticité du message.

2.2.3 Synthèse des approches actuelles

Le tableau 1 résume les travaux analysés et situe Nova par rapport à eux. Les colonnes correspondent à des propriétés architecturales (de conception), non à des mesures de performance ; ces dernières sont traitées au chapitre 6.

TABLE 1 – Comparatif des travaux existants et positionnement de la solution proposée

Travaux / Système	IA Edge	Modulaire	Temps réel	Hors ligne	Multi-modes
YOLOv8 XR [1]	✓	×	×	×	×
LLM-Glasses [2]	×	×	×	×	×
Vision stéréo	✓	×	×	✓	×
Hybride (ultrasons + cam.)	✓	×	✓	✓	×
Nova (proposé)	✓	✓	✓	✓	✓

✓ = présent × = absent ou non traité

Les briques individuelles sont donc matures, mais leur intégration dans une architecture modulaire, à faible latence et hors ligne reste un défi ouvert.

2.3 ANALYSE COMPARATIVE

2.3.1 Performance et latence

La performance d'un système d'assistance se juge d'abord à sa latence bout-en-bout (E2E) : le délai entre un événement physique (un obstacle qui entre dans le champ) et le retour à l'utilisateur (vibration, son). C'est la métrique la plus critique, car elle conditionne sa sécurité.

La littérature révèle un déficit général sur cet indicateur. Le système YOLOv8-XR affiche 583 ms de latence E2E. Les architectures à LLM distant [2] montent à 2–5 secondes. Seules les approches hybrides ultrasoniques passent sous 200 ms, au prix d'une sémantique pauvre (présence ou absence d'obstacle, sans identification).

Nova est conçu pour rester sous le seuil de 200 ms grâce à trois choix architecturaux : les capteurs HC-SR04 sont lus par l'Arduino sans mobiliser le processeur du Pi, la communication entre le Pi et le broker MQTT est purement locale, et la logique d'alerte est de complexité $\mathcal{O}(1)$. La surcharge du bus MQTT en local a été mesurée comme inférieure à 1 ms (voir chapitre 6) ; la latence bout-en-bout complète y est également reportée.

2.3.2 Complexité architecturale

Les architectures à serveurs distants ou à grands modèles de langage [2] cumulent une forte complexité logicielle et réseau. La dépendance à des API tierces (GPT-4o, services de reconnaissance vocale en ligne) entraîne des coûts variables, un risque de discontinuité de service et une exigence de connectivité incompatible avec un usage autonome.

À l'opposé, les systèmes purement locaux (vision stéréo, ultrason simple) sont peu complexes mais sémantiquement pauvres : ils détectent des présences, pas des classes d'objets ni du texte. Nova cherche un équilibre : ses modules s'exécutent localement (MediaPipe, Vosk, Tesseract), et le bus MQTT contient la complexité d'intégration.

2.3.3 Robustesse et fiabilité

La robustesse est la capacité à fonctionner en conditions dégradées (faible luminosité, panne d'un composant, surcharge processeur). Les systèmes monolithiques sont vulnérables : une panne du module de détection arrête tout. Les systèmes hybrides multi-capteurs gagnent en redondance, mais seulement au niveau matériel ; une panne logicielle reste fatale.

Par conception, l'architecture publish/subscribe de Nova apporte une robustesse logicielle : chaque module est un processus indépendant. L'arrêt du module Vision n'interrompt pas le module Navigation, car le broker absorbe la déconnexion. La validation expérimentale de cette isolation (arrêt forcé d'un module en cours d'exécution) est présentée au chapitre 6.

2.3.4 Synthèse comparative

Le tableau 2 confronte les quatre familles d'approches. Pour Nova, les cellules indiquent les propriétés de conception ; les valeurs chiffrées correspondantes sont établies au chapitre 6.

2.4 LIMITES IDENTIFIÉES

2.4.1 Insuffisances méthodologiques

La première lacune est la **rigidité architecturale**. Chaque système analysé vise une tâche précise : détection d'obstacles *ou* raisonnement contextuel *ou* guidage haptique, rarement les trois de façon

TABLE 2 – Comparaison critique des approches assistives

Critère	YOLO embarqué [1]	LLM distant [2]	Hybride capteurs	Nova (proposé)
Performance	Précision correcte, E2E 583 ms	Sémantique riche, 2–5 s	<200 ms, sémantique faible	Cible <200 ms, sémantique complète par modules
Complexité	Modérée (1 modèle)	Très élevée (API distante)	Faible	Faible à modérée (MQTT local)
Robustesse	Panne = arrêt total	Dépendante du cloud	Bonne au niveau matériel	Isolation des pannes par module
Hors ligne	Partiel	Non	Oui	Oui (intégral)
Modulaire	Non	Non	Non	Oui (greffons à chaud)
Multi-usages	Non	Limité	Non	Oui (profils configurables)

orchestrée et reconfigurable. Ajouter une fonction (remplacer un module de lecture de texte par de la reconnaissance faciale, par exemple) impose, dans ces architectures, de réécrire tout ou partie du pipeline.

À cette rigidité s’ajoute une **absence de personnalisation**. Aucun des systèmes étudiés ne propose de profils utilisateurs permettant d’ajuster les seuils d’alerte, le canal de retour (audio ou haptique) ou la priorité des modules selon le contexte et les préférences.

2.4.2 Faiblesses expérimentales

Un écart systématique sépare les évaluations de laboratoire de l’usage réel. Les jeux de données d’entraînement sont souvent génériques (COCO, ImageNet), peu représentatifs des environnements urbains des pays en développement. Les évaluations privilégient les métriques techniques (mAP, FPS, WER) et négligent l’expérience réelle : un système qui multiplie les alertes sonores peut, paradoxalement, alourdir la charge cognitive et réduire la sécurité d’une personne malvoyante.

La charge thermique est l’autre angle mort. Aucun des travaux analysés ne rapporte de mesures de température sous charge prolongée, alors que le *thermal throttling* du Raspberry Pi (chute de fréquence de 1 800 MHz à 800 MHz) apparaît dès la première heure d’inférence soutenue avec un modèle lourd comme YOLOv8n.

2.4.3 Contraintes techniques

LE DILEMME LATENCE-SÉMANTIQUE. Les modèles les plus riches sémantiquement (LLM, détecteurs généralistes) produisent des latences incompatibles avec une navigation sûre, tandis que les modèles à faible latence (CNN légers, ultrason) restent sémantiquement pauvres. Aucune solution existante ne tranche ce dilemme de façon satisfaisante dans les contraintes d’un dispositif porté autonome.

LA DÉPENDANCE À LA CONNECTIVITÉ. Les approches fondées sur le cloud ou sur des API de modèles de langage introduisent une vulnérabilité de fond : sans réseau, le système devient partiellement ou totalement inopérant. Pour un dispositif de sécurité destiné à des personnes

malvoyantes, susceptibles de se trouver en zone rurale, en sous-sol ou dans un bâtiment mal couvert, cette dépendance est inacceptable.

2.4.4 *Conclusion et nécessité d'une nouvelle approche*

Prises ensemble, ces limites montrent qu'améliorer un algorithme isolé ou remplacer un modèle par un plus récent ne suffit pas. La réponse doit être architecturale : un cadre où des modules interchangeables communiquent par un bus de messages léger, s'exécutent entièrement en local, et se configurent selon le profil de l'utilisateur.

Nova s'inscrit dans cet espace. La plateforme utilise MQTT comme bus interne, MediaPipe Hands comme moteur de vision léger pour les gestes, YOLOv11n pour la détection d'objets et Moondream 2 pour la compréhension de scène à la demande, le tout porté par une architecture à greffons qui autorise l'ajout de modules sans toucher au code central. L'objectif est de combler l'écart entre la maturité des briques prises isolément et le déploiement d'un système d'assistance complet, personnalisable et résilient.

MÉTHODOLOGIE

Ce chapitre décrit l'approche scientifique adoptée pour concevoir la plateforme Nova. Le fil conducteur est une démarche de *Design Science Research* : construire un artefact, mesurer ses performances réelles sur le prototype physique, puis réviser les choix au vu des données observées. Cette boucle a guidé deux décisions concrètes : écarter YOLOv8n au profit de MediaPipe *pour la tâche de reconnaissance gestuelle*, et abandonner `eventlet` pour le threading natif de Python. Chaque choix présenté ici remonte à une observation expérimentale, et non à la seule littérature.

3.1 ARCHITECTURE PROPOSÉE

3.1.1 Description globale du système

Nova est une plateforme d'assistance embarquée organisée en quatre couches : la couche Physique (perception et actionneurs embarqués), la couche Middleware (bus MQTT et transport asynchrone), la couche Traitement Edge AI (inférence et analyse multimodale) et la couche Application (supervision, profils et restitution contextuelle). Une interface web accessible en réseau local complète l'ensemble.

Le principe directeur est la séparation entre le **nœud de capture** (le casque porté) et l'**interface de supervision** (application web ou mobile distante). L'opérateur n'a pas d'écran : il reçoit l'information par retour vocal (TTS) et haptique (vibreurs). Le superviseur, lui, visualise le flux vidéo annoté depuis n'importe quel navigateur du réseau local.

Trois nœuds de traitement se partagent les responsabilités.

- **Raspberry Pi 4B** : unité centrale. Il exécute l'orchestrateur, le serveur Flask + Socket.IO, le service vision (MediaPipe), la reconnaissance vocale (Vosk) et la synthèse vocale (eSpeak-NG), ainsi que le broker Mosquitto.
- **Arduino Uno Rev3** : déchargement des capteurs temps réel. Il lit les HC-SR04, DHT11, MQ-3 et IR toutes les 10 ms et commande les vibreurs en PWM, sans mobiliser le processeur du Pi. Il communique avec le Pi par liaison UART (9 600 baud).
- **ESP32-CAM** : nœud vision secondaire. Il diffuse un flux MJPEG en Wi-Fi et peut lire des capteurs I2C locaux.

La figure 3.2 présente l'ensemble et ses interconnexions.

3.1.2 Flux de données

MODULE VISION. Chaque cycle de traitement suit ce pipeline :

1. Picamera2 capture une image BGR 640×480 via libcamera.
2. L'image est convertie BGR $\rightarrow$ RGB et transmise au graphe MediaPipe Hands.
3. MediaPipe retourne 21 repères 3D par main détectée.
4. Un classifieur géométrique associe ces repères à l'un des quatre gestes reconnus (tableau 4).
5. Le résultat JSON et l'image annotée sont publiés sur MQTT (`nova/camera/processed` et `nova/plugins/hand_control/data`) et diffusés via le flux MJPEG Flask sur le port 5050.

MODULE NAVIGATION. L'Arduino mesure la distance avec les deux HC-SR04 en multiplexage temporel (un capteur toutes les 10 ms, pour éviter les interférences acoustiques), encode le résultat

en JSON et l'envoi au Pi par UART. Le module Navigation lit ce flux, calcule le niveau d'alerte (équation 1) et publie sur nova/nav/alert. Le signal PWM est appliqué aux vibreurs gauche et droit proportionnellement à la distance, du côté de l'obstacle détecté.

Pour expliciter l'interconnexion physique et le partitionnement électronique de ce système cyber-physique, le schéma bloc matériel de la Figure 3.1 cartographie l'architecture matérielle distribuée de la plateforme.

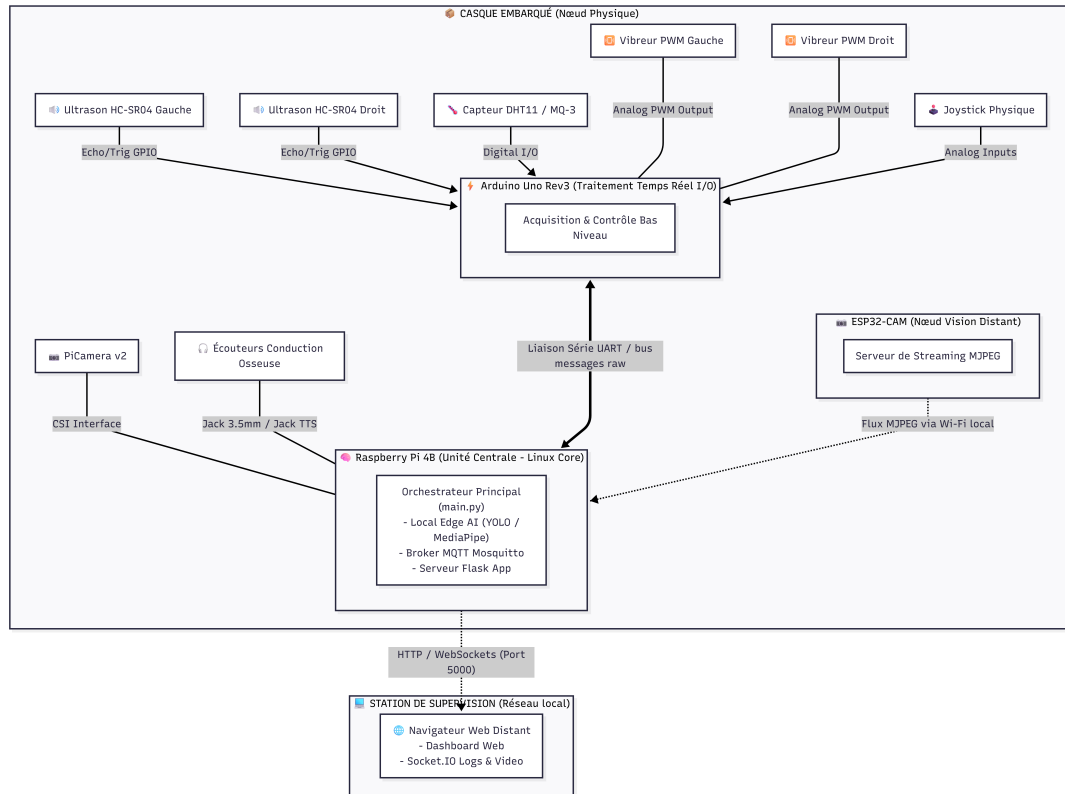


FIGURE 3.1 – Architecture matérielle distribuée de la plateforme Nova

3.1.3 Justification des choix architecturaux

Le patron publish/subscribe (MQTT)

Le choix d'un bus publish/subscribe via MQTT répond à trois besoins propres aux systèmes embarqués d'assistance.

1. **Découplage spatial et temporel.** Contrairement au modèle client-serveur (HTTP), le nœud de capture et le nœud de supervision n'ont pas à maintenir une connexion synchrone directe. Le traitement reste fluide même lors de micro-coupures réseau.
2. **Faible surcharge.** Pour transmettre en continu les coordonnées des repères de la main, l'en-tête réduit de MQTT pèse moins que l'équivalent JSON sur HTTP. La surcharge du bus en local a été mesurée à 0,32 ms en moyenne (cf. chapitre 6), ce qui valide ce choix.
3. **Extensibilité.** Ajouter un abonné (un retour haptique, une interface web) ne demande aucune modification du producteur de données (la caméra).

MediaPipe plutôt que YOLOv8n pour les gestes

Ce choix ne relève pas du principe : il découle d'un échec observé lors des premiers tests. Exécuté en continu pour la reconnaissance gestuelle, YOLOv8n saturait le processeur du Pi et déclenchait le *thermal throttling* (chute de fréquence de 1 800 MHz à 800 MHz), ne laissant presque

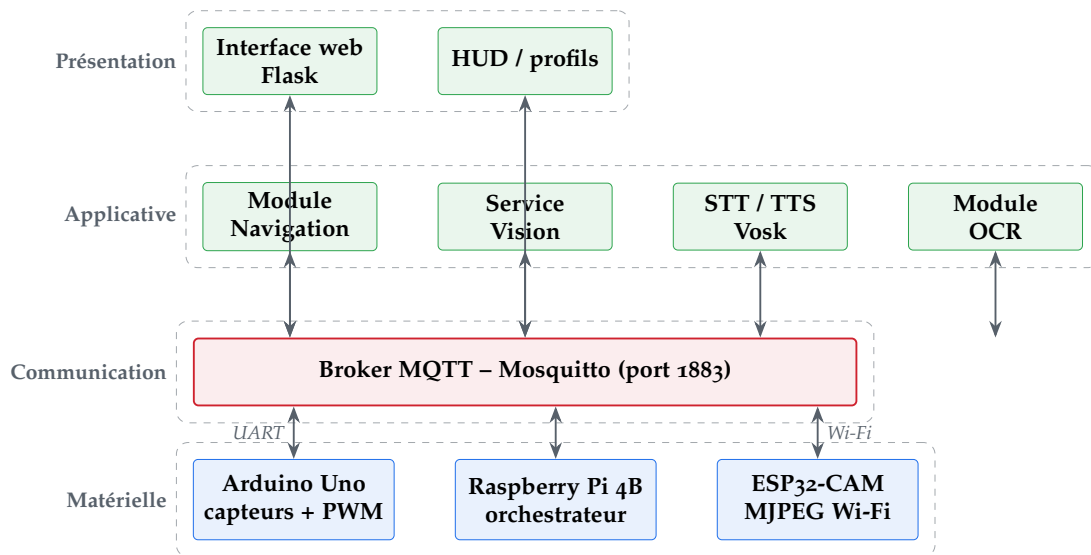


FIGURE 3.2 – Architecture globale de la plateforme Nova

aucune ressource aux autres modules. MediaPipe Hands, conçu pour l'extraction de repères, libère assez de processeur pour faire tourner Flask, MQTT, la reconnaissance et la synthèse vocale en parallèle.

Le choix de MediaPipe Hands pour la reconnaissance gestuelle a été motivé par des critères de légèreté et de latence démontrés dans la littérature. MediaPipe Hands, conçu spécifiquement pour l'extraction de repères 3D sur matériel embarqué, présente une latence d'inférence de 28 à 55 ms sur Raspberry Pi 4B [4], contre 120 à 200 ms pour un détecteur YOLOv8n PyTorch complet, avec une empreinte mémoire nettement inférieure. Ce choix permet de libérer des ressources CPU pour les modules Flask, MQTT, STT et TTS fonctionnant en parallèle, conformément à l'objectif de modularité du système. Une comparaison chiffrée détaillée sur notre configuration sera menée lors de la campagne de validation sur Raspberry Pi.

Flask + Socket.IO en mode *threading*

La version initiale utilisait *eventlet*. Ce moteur asynchrone applique un *monkey-patching* des entrées/sorties Python qui entre en conflit avec les bibliothèques C++ de Picamera2 et d'OpenCV : le thread de capture se figeait toutes les 3 à 8 minutes. Après passage à `async_mode='threading'`, le blocage périodique a disparu lors des sessions de validation courtes réalisées pendant l'intégration du prototype.

3.1.4 Analyse fonctionnelle (UML)

Afin de structurer le comportement de la plateforme du point de vue de l'utilisateur final et du superviseur, la Figure 3.3 détaille le diagramme de cas d'utilisation globale de Nova.

3.1.5 Architecture et Modélisation Logicielle

La conception du micrologiciel s'appuie sur le paradigme orienté objet pour garantir l'interopérabilité des briques de vision et de navigation. La Figure 3.4 présente le diagramme de classes qui régit le script central `main.py` et ses gestionnaires périphériques.

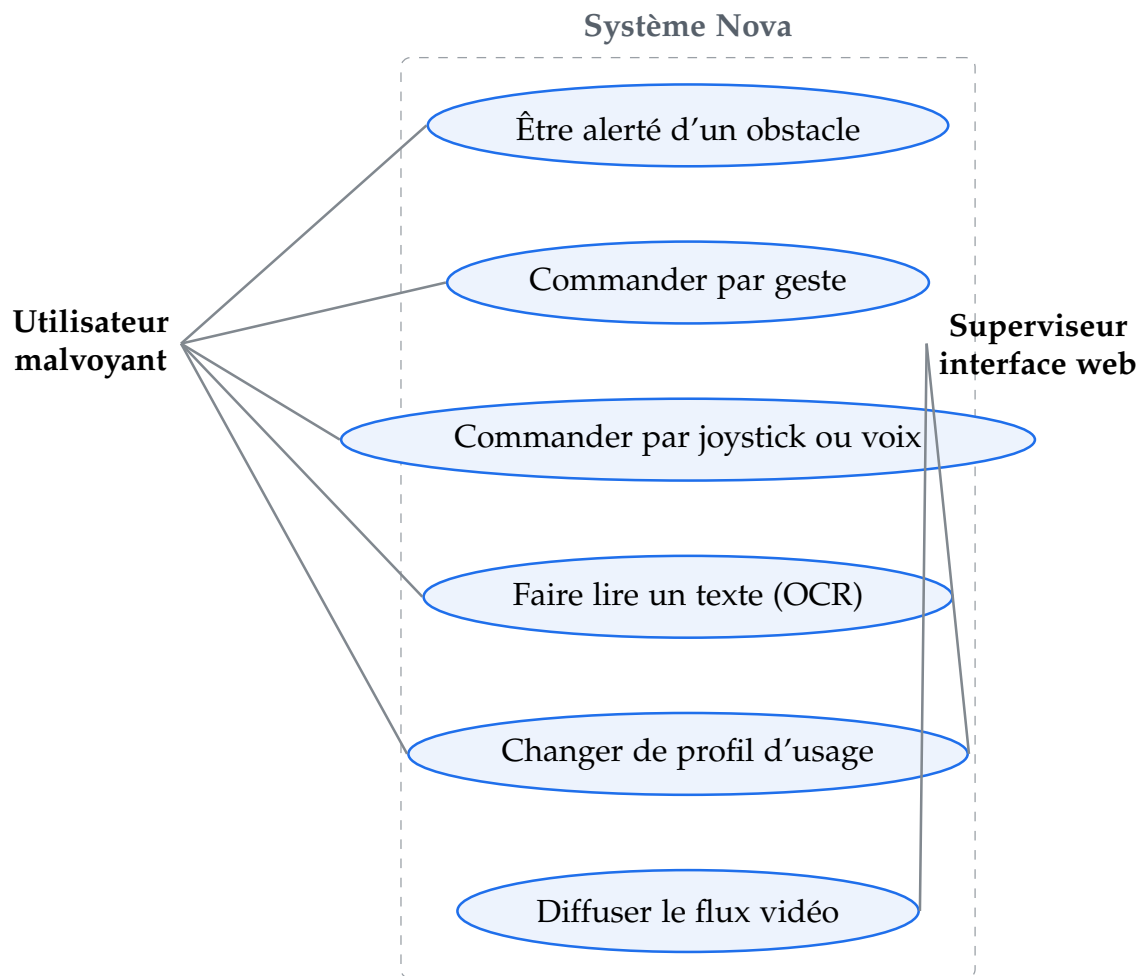


FIGURE 3.3 – Diagramme de cas d'utilisation globale du système Nova

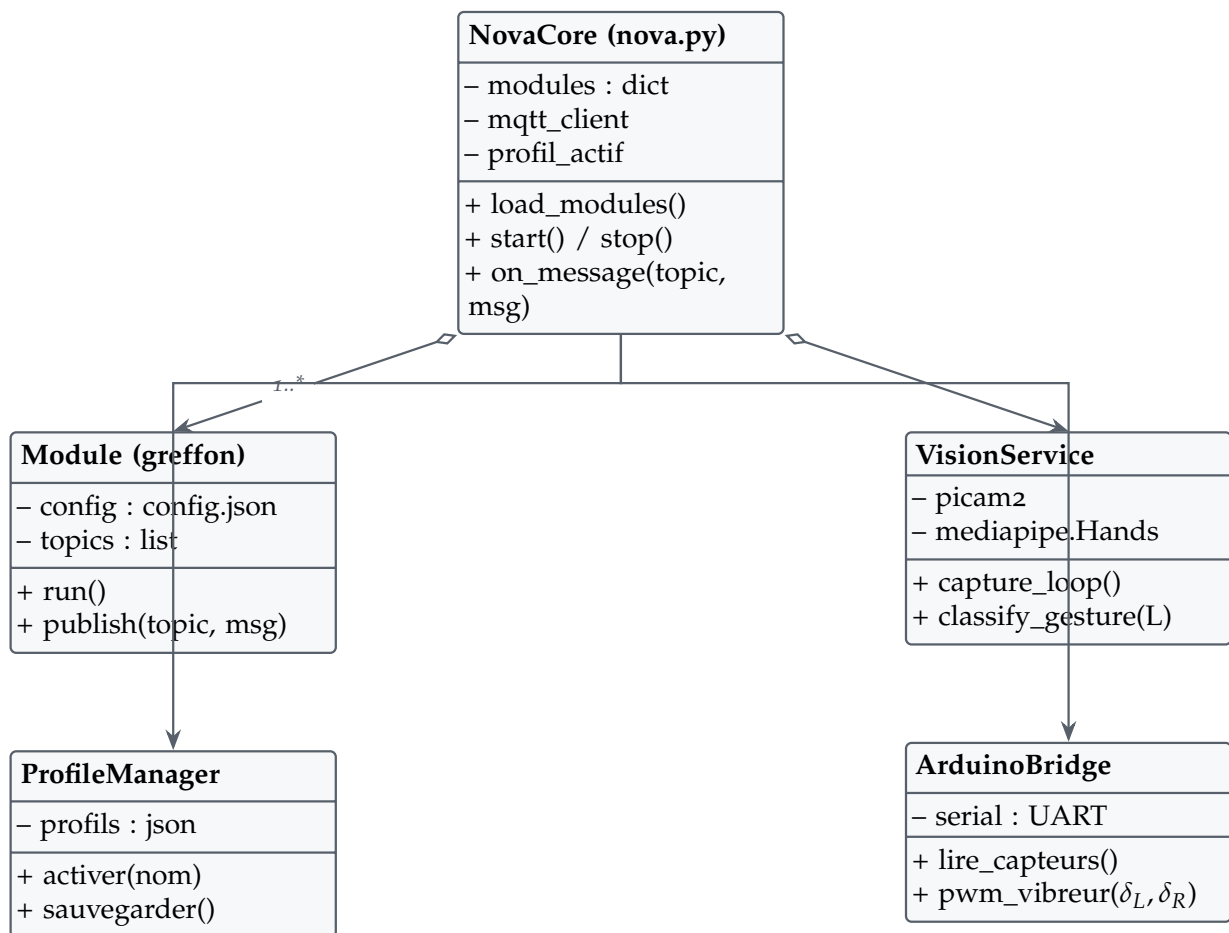


FIGURE 3.4 – Diagramme de classes de l’application centrale de Nova

3.2 MODÉLISATION

3.2.1 Formulation mathématique

MODÈLE D'ALERTE D'OBSTACLE. La distance d mesurée par le HC-SR04 est convertie en un rapport cyclique PWM δ appliqué aux vibreurs :

$$\delta(d) = \begin{cases} 0, & \text{si } d > d_{\text{loin}}, \\ 50 + 50 \frac{d_{\text{loin}} - d}{d_{\text{loin}} - d_{\text{proche}}}, & \text{si } d_{\text{proche}} < d \leq d_{\text{loin}}, \\ 100, & \text{si } d \leq d_{\text{proche}}. \end{cases} \quad (1)$$

où $d_{\text{loin}} = 150$ cm et $d_{\text{proche}} = 80$ cm sont des seuils configurables dans le profil utilisateur. L'intensité haptique progresse linéairement entre les deux seuils, offrant un retour proportionnel à l'imminence du danger.

3.2.2 Hypothèses de travail

1. Le réseau est un LAN Wi-Fi local sans accès internet : toute la chaîne IA (MediaPipe, Vosk, Tesseract) fonctionne hors ligne.
2. Lors d'une commande gestuelle, la main de l'opérateur est visible dans le champ de la caméra (distance typique 30 à 80 cm).
3. L'éclairage ambiant est suffisant pour MediaPipe ; sous ce seuil, la commande au joystick et le module Navigation ultrasonique prennent le relais.
4. Les obstacles sont des surfaces solides (murs, meubles, personnes) ; les HC-SR04 ne détectent ni les obstacles transparents ni les surfaces très absorbantes.

3.2.3 Variables du système

TABLE 3 – Variables principales du modèle système Nova

Variable	Type	Description
d_L, d_R	Float (cm)	Distance obstacle gauche / droite
δ_L, δ_R	Int [0..100]	Rapport cyclique PWM vibreur G / D
g	String	Geste : START_SYSTEM, STOP_SYSTEM, READ_TEXT, TRIGGER_MOONDREAM
c_g	Float [0..1]	Confiance du geste (éq. 2)
f	Float (FPS)	Cadence du service vision
T	Float (°C)	Température ambiante (DHT11)
H	Float (%)	Humidité ambiante (DHT11)
CPU	Float (%)	Utilisation processeur (psutil)
RAM	Float (Mo)	Mémoire des modules actifs

3.3 ALGORITHMES

3.3.1 Classification des gestes (MediaPipe)

DESCRIPTION. Chaque image est traitée par MediaPipe Hands, qui retourne 21 repères (x, y, z) normalisés. L'algorithme exploite la coordonnée y (axe vertical, croissant vers le bas) et x (axe horizontal) pour déterminer l'état levé ou replié de chaque doigt, puis associe la configuration binaire à l'un des quatre gestes du tableau 4. Le pouce est traité sur l'axe x , car il se déplace latéralement et non verticalement.

TABLE 4 – Bibliothèque des quatre gestes implémentés dans `hand_control/main.py`

Geste	Doigts [P,I,M,A,Au]	Commande MQTT	Action
Pleine main	[T, T, T, T, T]	START_SYSTEM	Démarrage de la plateforme
Poing	[*, F, F, F, F]	STOP_SYSTEM	Arrêt propre des modules
Index seul	[F, T, F, F, F]	READ_TEXT	Lecture de texte (OCR Tesseract)
Pouce seul	[T, F, F, F, F]	TRIGGER_MOONDREAM	Analyse de scène (Moondream 2)

P = pouce, I = index, M = majeur, A = annulaire, Au = auriculaire. T = levé, F = replié, * = non discriminant.

Algorithm 1 Classification de gestes par analyse des repères MediaPipe

Require: Image BGR F , repères $L[0..20]$, seuil $\tau = 0.8$ s

Ensure: Geste confirmé g^* ou $\emptyset$

Convertir F en RGB $\rightarrow F'$

$L \leftarrow \text{MediaPipe.Hands.process}(F')$

if L est vide **then**

return $\emptyset$

end if

$fingers \leftarrow []$ {Pouce : axe horizontal}

if $L[4].x > L[2].x + 0.02$ **then**

$fingers.append(\text{True})$

else

$fingers.append(\text{False})$

end if{Index, majeur, annulaire, auriculaire : axe vertical}

for $tip \in [8, 12, 16, 20]$ **do**

$fingers.append(L[tip].y < L[tip - 2].y)$

end for

$g \leftarrow \text{MapGeste}(fingers)$ {Confirmation temporelle}

if $g \neq g_{\text{prev}}$ **then**

$g_{\text{prev}} \leftarrow g$; $t_0 \leftarrow \text{temps}()$

else

if $\text{temps}() - t_0 \geq \tau$ **then**

 Publier MQTT : $\{gesture: g, confidence: c_g\}$

$t_0 \leftarrow \text{temps}()$

end if

end if

PSEUDOCODE. La confirmation temporelle c_g est définie par :

$$\begin{aligned}
c_g(t) &= \frac{t - t_0}{\tau}, \\
g^* &= \begin{cases} g, & \text{si } c_g(t) \geq 1 \text{ et } g = g_{\text{prev}}, \\ \emptyset, & \text{sinon.} \end{cases}
\end{aligned} \tag{2}$$

Un geste n'est publié que s'il est maintenu sans interruption pendant au moins $\tau = 0.8$ s. Ce filtre réduit nettement les faux positifs ; la réduction mesurée est reportée au chapitre 6

L'implémentation logicielle de la classification des gestes et de la confirmation temporelle est détaillée dans le script `hand_control/main.py`. Le Listing 3.1 présente l'extrait de code correspondant à cette logique :

Listing 3.1 – Classification des gestes et confirmation temporelle

```

1 lms = hand_landmarks.landmark
2 fingers = []
3
4 # Pouce : axe horizontal (anatomie laterale)
5 if lms[4].x > lms[2].x + 0.02:
6     fingers.append(True)
7 else:
8     fingers.append(False)
9
10 # Index (8), Majeur (12), Annulaire (16), Auriculaire (20)
11 for tip in [8, 12, 16, 20]:
12     fingers.append(lms[tip].y < lms[tip - 2].y)
13
14 # Mapping gestes
15 if fingers == [True, True, True, True, True]:
16     current_gesture = "START_SYSTEM"
17 elif fingers[1:] == [False, False, False, False]:
18     current_gesture = "STOP_SYSTEM"
19 elif fingers == [False, True, False, False, False]:
20     current_gesture = "READ_TEXT"
21 elif fingers == [True, False, False, False, False]:
22     current_gesture = "TRIGGER_MOONDREAM"
23
24 # Confirmation temporelle (seuil : 0.8 s)
25 if current_gesture != self.last_gesture:
26     self.last_gesture = current_gesture
27     self.gesture_start_time = time.time()
28 else:
29     elapsed = time.time() - self.gesture_start_time
30     if elapsed >= self.confirm_threshold:
31         payload = json.dumps({
32             "gesture": current_gesture,
33             "confidence": elapsed / self.confirm_threshold,
34             "timestamp": time.time()
35         })
36         self.client.publish(TOPIC_RESULTS, payload)
37         self.gesture_start_time = time.time()

```

COMPLEXITÉ. La classification est $\mathcal{O}(1)$: cinq comparaisons scalaires, indépendantes du nombre d'images. La latence dominante est celle de l'inférence MediaPipe. La confirmation temporelle ajoute un filtre applicatif de 0,8 s, pas un coût algorithmique.

3.3.2 Détection d'obstacles

DESCRIPTION. L'Arduino lit les deux HC-SR04 en multiplexage temporel (10 ms par capteur), encode les distances en JSON et les transmet par UART. Le module Navigation reçoit ce flux, applique la formule (1) et publie l'alerte sur MQTT.

Listing 3.2 – Calcul d'alerte et commande PWM — navigation/main.py

```

1 def compute_alert(distance_cm: float,
2                   d_loin: float = 150.0,
3                   d_proche: float = 80.0) -> dict:
4     if distance_cm > d_loin:
5         return {"level": "CLEAR", "duty": 0, "tts": ""}
6     elif distance_cm > d_proche:
7         duty = int(50 + (d_loin - distance_cm)
8                   / (d_loin - d_proche) * 50)
9         return {"level": "MEDIUM", "duty": duty,
10                "tts": f"Obstacle a {distance_cm:.0f} cm"}
11     else:
12         return {"level": "HIGH", "duty": 100,
13                "tts": "Danger, obstacle tres proche"}

```

COMPLEXITÉ. La fonction est $\mathcal{O}(1)$: deux comparaisons et une multiplication. La latence bout-en-bout du module (UART Arduino $\rightarrow$ MQTT $\rightarrow$ PWM vibreur) est mesurée et analysée au chapitre 6 ; l'objectif de conception est de rester sous 200 ms.

3.3.3 Workflow dynamique du système : cycle de vie d'un événement critique

Afin d'illustrer la synergie entre les quatre couches de l'architecture, la cinématique d'un événement critique tel que la détection d'un obstacle immédiat suivie d'un retour haptique et d'un profilage contextuel suit un flux de données asynchrone et non-bloquant décrit ci-après :

- **Étape 1 : Échantillonnage et Filtrage (Couche Physique)**

Le microcontrôleur Arduino Uno interroge de manière autonome le capteur ultrasonique HC-SR04 toutes les 10 ms. Dès qu'une variation significative de distance est mesurée, la valeur brute est encapsulée.

- **Étape 2 : Ingestion et Routage (Couche Middleware)**

La trame brute est transmise via la liaison UART au Raspberry Pi, où elle est immédiatement publiée sur le topic MQTT nova/sensor/ultrasonic sous forme de message JSON horodaté. Le broker dispatche l'information instantanément aux modules abonnés.

- **Étape 3 : Inférence et Analyse Multimodale (Couche Traitement Edge AI)**

Le module de navigation souscrit au topic, extrait la distance et calcule l'indice de criticité en temps réel ($\mathcal{O}(1)$). Parallèlement, le pipeline vision analyse le flux de la caméra pour confirmer l'absence de commandes de contournement manuelles (via l'analyse des gestes par MediaPipe).

- **Étape 4 : Prise de Décision Contextuelle (Couche Application)**

L'orchestrateur central interroge l'état global du système (profil utilisateur actif et données de luminosité de la LDR). Si les conditions de visibilité sont dégradées, le système adapte dynamiquement la priorité des retours d'information.

- **Étape 5 : Restitution Matérielle et Télémétrie (Retour aux Couches 1 et 4)**

Une commande d'alerte finale est publiée sur le topic nova/nav/alert. L'Arduino intercepte ce message et déclenche instantanément la vibration des moteurs via un signal PWM proportionnel au danger. Simultanément, la console de supervision distante est mise à jour via WebSockets.

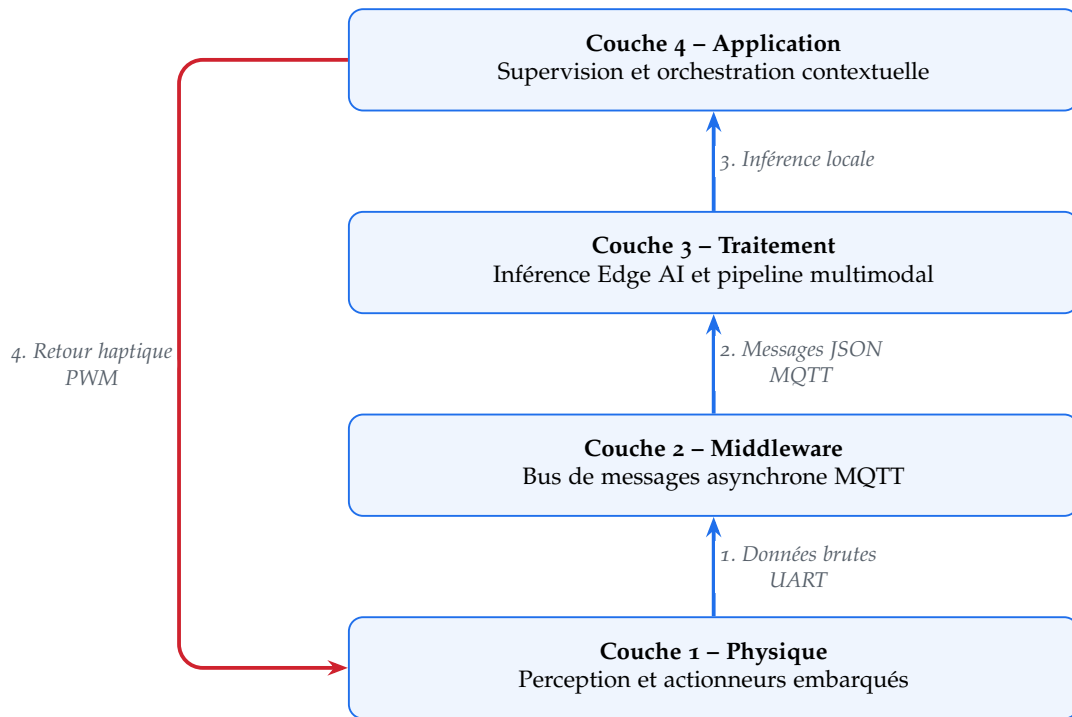


FIGURE 3.5 – Architecture fonctionnelle Nova : flux dynamique

3.3.4 Chargement dynamique des modules

DESCRIPTION. Au démarrage de `nova.py`, la fonction `load_modules()` parcourt le répertoire `plugins/modules/`, lit le `config.json` de chaque sous-dossier et construit le dictionnaire `AVAILABLE_MODULES`. Ajouter un module revient donc à déposer son dossier et son `config.json`, sans modifier une ligne du code principal.

Listing 3.3 – Chargement dynamique des modules `nova.py`

```

1 def load_dynamic_modules():
2     global AVAILABLE_MODULES
3     for folder in os.listdir(PLUGINS_BASE_DIR):
4         folder_path = PLUGINS_BASE_DIR / folder
5         if folder_path.is_dir():
6             config_file = folder_path / "config.json"
7             if config_file.exists():
8                 with open(config_file, 'r', encoding='utf-8') as f:
9                     AVAILABLE_MODULES[folder] = json.load(f)
10                    logger.info(f"Module charge : {folder}")

```

COMPLEXITÉ. $\mathcal{O}(n)$, où n est le nombre de sous-dossiers dans `plugins/modules/`. Exécutée une seule fois au démarrage, l'opération est négligeable. Elle garantit qu'un module futur s'intègre sans recompilation ni modification du cœur applicatif.

CHOIX TECHNIQUES

Ce chapitre détaille l’architecture logicielle et matérielle du système, en mettant en évidence les technologies sélectionnées pour le casque intelligent. Il justifie également les décisions de conception prises pour répondre aux exigences fonctionnelles du projet, tout en respectant les contraintes strictes inhérentes aux systèmes embarqués d’assistance à la navigation.

4.1 TECHNOLOGIES UTILISÉES

Le système s’articule autour d’une pile technologique optimisée pour l’Edge Computing, combinant des modèles d’intelligence artificielle légers, des protocoles de communication IoT et un serveur web embarqué :

— **Vision par Ordinateur et Intelligence Artificielle :**

- **MediaPipe** : Utilisé pour le traitement ultra-rapide des flux vidéo et l’extraction de repères spatiaux de base.
- **YOLOv11n (Nano)** : Modèle de détection d’objets de pointe, sélectionné pour sa légèreté et sa précision sur du matériel aux ressources limitées.
- **Moondream 2** : Modèle de vision-langage (VLM) compact, intégré pour la reconnaissance avancée du contexte et la description sémantique de l’environnement (analyse de scènes complexes).

— **Backend et Serveur Web :**

- **Python** : Langage principal pour l’orchestration des modèles d’IA et la logique métier.
- **Flask** : Micro-framework web gérant l’interface d’administration et les routes locales.

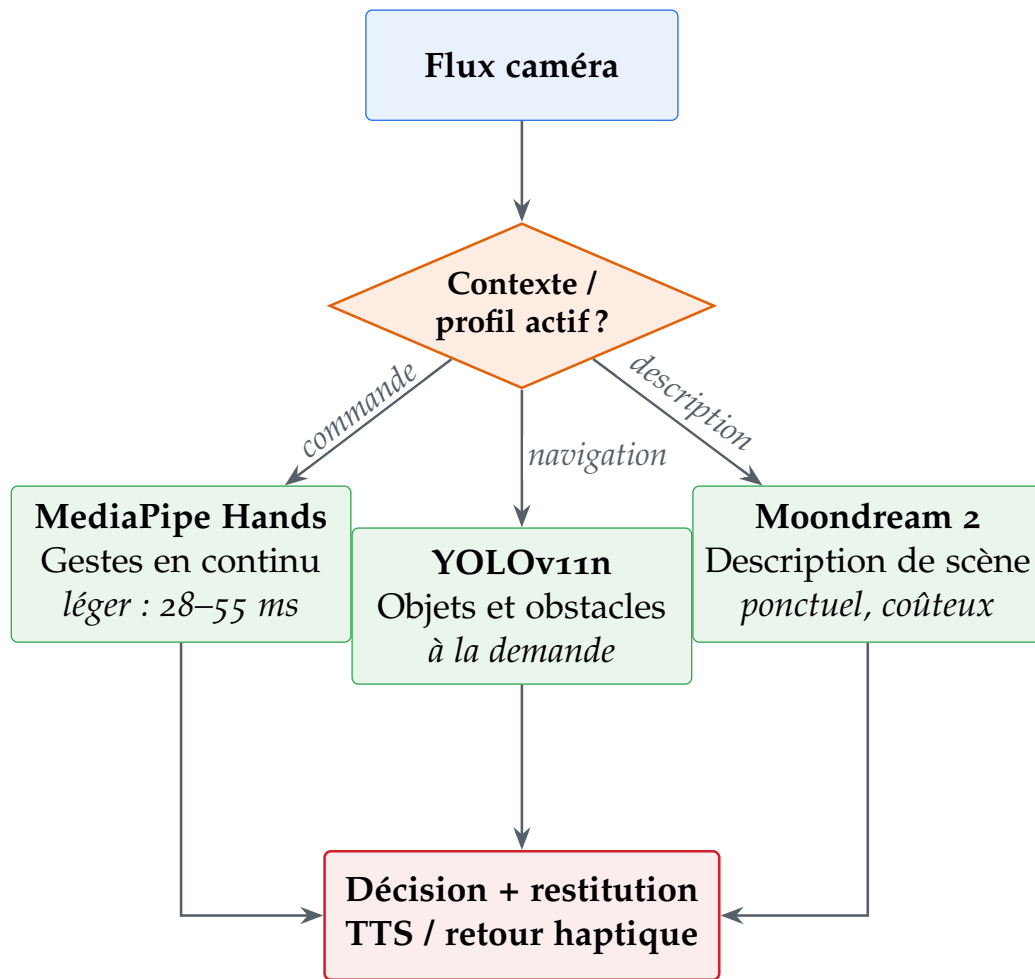
— **Communication et IoT :**

- **MQTT (Message Queuing Telemetry Transport)** : Protocole de messagerie central pour l’échange de données entre les capteurs matériels, l’unité de traitement et le retour utilisateur.

4.2 JUSTIFICATION DES CHOIX DE CONCEPTION

Architecture IA : La synergie MediaPipe, YOLOv11n et Moondream 2

Plutôt que de s’appuyer sur un modèle monolithique lourd, l’approche architecturale retenue consiste à utiliser un pipeline modulaire. MediaPipe agit comme un filtre de première ligne, capable de traiter les flux vidéo avec une latence quasi-nulle. Lorsqu’une analyse plus sémantique est requise, YOLOv11n prend le relais pour détecter et classifier précisément les obstacles. Enfin, l’intégration de Moondream 2 permet d’apporter une dimension cognitive au système : au lieu de simplement signaler un objet de manière isolée, le système peut analyser la scène globale pour offrir un contexte de navigation enrichi à l’utilisateur. Cette séparation des tâches garantit que les calculs lourds ne sont déclenchés que lorsque cela est strictement nécessaire, préservant ainsi les ressources du système embarqué. La Figure 4.1 illustre cette cascade décisionnelle. Ce partitionnement garantit que le processeur n’alloue ses ressources maximales qu’en cas de nécessité absolue, répondant ainsi directement à la contrainte de temps réel et d’économie d’énergie.



Contrainte embarquée : deux modèles simultanés maximum sur Raspberry Pi 4B

FIGURE 4.1 – Synergie et déclenchement conditionnel des modèles d'IA embarqués

Serveur Backend : Flask vs FastAPI

Bien que FastAPI soit aujourd'hui réputé pour ses performances supérieures et sa gestion native de l'asynchronisme, le choix s'est porté sur **Flask** pour l'architecture de ce prototype. Dans le contexte de notre système, la majorité des traitements vidéo (OpenCV, MediaPipe) sont intrinsèquement synchrones. Flask offre une simplicité d'implémentation et une stabilité éprouvée lors de la manipulation de flux vidéo directs via des générateurs (streaming). La charge web attendue reste faible : une interface locale de supervision, un flux MJPEG et quelques événements Socket.IO. La campagne de mesure du bus local, présentée au chapitre 6, indique par ailleurs une latence médiane MQTT de 1,82 ms sans perte observée ; le goulot d'étranglement mesuré se situe donc dans le pipeline vision et la charge CPU, non dans le choix du micro-framework web. Flask est ainsi retenu comme orchestrateur simple et suffisamment robuste pour un backend local, plutôt que comme solution destinée à absorber un trafic web massif.

Communication : Le choix de MQTT face aux WebSockets

Pour la transmission des données de navigation et des alertes capteurs, le protocole **MQTT** a été privilégié par rapport aux WebSockets traditionnels. Les WebSockets établissent une connexion TCP persistante et bidirectionnelle, ce qui génère un surcoût de données (*overhead*) non négligeable et demande des ressources continues pour maintenir la connexion active. À l'inverse, MQTT est un protocole de type *Publish/Subscribe* conçu spécifiquement pour l'Internet des Objets (IoT). Il présente des avantages décisifs pour ce projet :

1. **Légereté de l'en-tête** : Les paquets MQTT sont extrêmement petits par rapport au protocole HTTP/WebSockets, ce qui réduit la bande passante nécessaire.
2. **Gestion de l'instabilité** : MQTT gère nativement les micro-coupures de connexion entre les composants matériels, très fréquentes sur un dispositif portable en mouvement.
3. **Qualité de Service (QoS)** : MQTT permet de définir des niveaux de priorité. Une alerte d'obstacle imminent peut être configurée avec un QoS élevé (garantie de livraison), assurant la sécurité absolue de l'utilisateur.

4.3 ARCHITECTURE SYSTÈME EN QUATRE COUCHES

Pour garantir le découplage, la modularité et la tolérance aux pannes sur un matériel contraint, l'architecture logicielle et matérielle de Nova a été restructurée en quatre couches distinctes, rompant avec les approches monolithiques traditionnelles :

1. **Couche Physique (Perception et Action)** : Située au plus près du matériel, elle englobe l'unité microcontrôleur (Arduino Uno) chargée de l'acquisition haute fréquence des capteurs environnementaux (HC-SR04, DHT11, MQ-3) et du pilotage direct des actionneurs (vibreurs haptiques en PWM, avertisseur sonore). Elle inclut également les périphériques d'acquisition multimédia (Caméra Pi, microphone) connectés au processeur central.
2. **Couche Middleware (Communication et Transport)** : Cette couche assure le transport asynchrone et le routage des messages via le protocole léger MQTT (géré par le broker Mosquitto) et une liaison série (UART) dotée de convertisseurs de niveaux logiques. Elle standardise les échanges sous forme de trames JSON et isole complètement la capture des données de leur traitement métier.
3. **Couche Traitement Edge AI (Logique Métier et Inférence)** : Exécutée intégralement en local sur le Raspberry Pi 4B, cette couche héberge l'orchestrateur principal (`nova.py`) et gère le cycle de vie des pipelines d'intelligence artificielle (MediaPipe pour la détection de repères gestuels, YOLOv11n pour l'évitement d'obstacles, et Vosk pour la reconnaissance vocale automatique).

4. **Couche Application (Supervision, Contexte et Restitution) :** Couche supérieure responsable de la gestion des profils utilisateurs, du serveur web embarqué (Flask + Socket.IO) et du tableau de bord de télémétrie destiné à un superviseur distant. C'est elle qui contextualise les décisions (par exemple, privilégier le retour vocal si la luminosité captée par la LDR est nulle).

4.4 CONTRAINTES PRISES EN COMPTE

La conception globale du prototype a été façonnée par quatre contraintes majeures :

1. **Contrainte de connectivité (Fonctionnement hors ligne obligatoire) :** L'utilisateur doit pouvoir se déplacer en toute sécurité dans n'importe quel environnement, y compris dans des zones dépourvues de couverture réseau (zones blanches, sous-sols). Par conséquent, le système est conçu pour l'Edge Computing : l'intégralité du traitement de l'intelligence artificielle est exécutée localement sur l'unité de calcul embarquée, sans aucune dépendance au Cloud.
2. **Contrainte de temps réel (Latence < 200 ms) :** Pour l'assistance à la navigation, un délai de traitement trop long rendrait le système inutilisable, voire dangereux. L'architecture logicielle, le choix de modèles d'IA optimisés et l'utilisation de MQTT ont été calibrés pour garantir que le délai entre la détection d'un obstacle physique et le retour d'information à l'utilisateur reste strictement sous la barre des 200 millisecondes.
3. **Contrainte de consommation électrique :** S'agissant d'un dispositif *wearable* (portable), l'autonomie est primordiale. Les composants matériels et les choix algorithmiques ont été sélectionnés pour offrir le meilleur ratio performance/consommation d'énergie, évitant l'usage de composants trop énergivores pour privilégier des processeurs optimisés pour l'embarqué.
4. **Contrainte économique (Budget plafonné) :** Pour que cette technologie d'assistance reste accessible, le coût de production du prototype a été fixé autour de 250 dollars. Cette contrainte financière justifie l'utilisation de composants grand public et de modules open-source, prouvant qu'il est possible de créer un dispositif d'assistance de haute technologie à un coût maîtrisé.

IMPLÉMENTATION

Ce chapitre décrit la mise en œuvre concrète du système Nova. Il couvre la configuration matérielle précise du prototype, l'organisation logicielle du projet, les bibliothèques utilisées, ainsi que les jeux de données qui ont servi à l'évaluation et à l'entraînement des modèles.

5.1 ENVIRONNEMENT EXPÉRIMENTAL

5.1.1 Configuration matérielle

Le prototype Nova est constitué de composants répartis en cinq familles fonctionnelles : unités de traitement, capteurs, modules d'interface, actionneurs et composants passifs. Chacun est décrit ci-dessous avec ses caractéristiques techniques clés et son rôle précis dans le système.

Unités de traitement et intelligence



FIGURE 5.1 – Masque de protection servant de châssis porteur au système Nova

MASQUE DE PROTECTION : CHÂSSIS DU CASQUE NOVA. Le masque de protection constitue le support physique sur lequel sont fixés les composants du casque : caméra frontale, capteurs ultrasoniques, récepteurs infrarouges latéraux et vibreurs. Sa monture souple épouse le contour du visage et répartit le poids de l'électronique embarquée, tandis que sa large surface frontale offre une zone de fixation stable et dégagée pour le module de vision. La sangle élastique réglable assure le maintien du dispositif quelle que soit la morphologie de l'utilisateur, et les ouïes de ventilation moulées dans la monture limitent la condensation et participent à la dissipation thermique passive des composants fixés à proximité. La masse du masque nu et celle du masque équipé n'ont pas été retenues comme indicateurs chiffrés dans cette version finale, faute de pesée instrumentée reproductible. L'évaluation porte donc sur les dimensions, la fixation, l'équilibrage et les conditions d'intégration mécanique. **Conditions optimales d'utilisation :** la fixation des composants doit préserver l'équilibre des masses entre l'avant (caméra, capteurs) et l'arrière (sangle), afin d'éviter le glissement du masque pendant la marche. Le câblage vers l'unité centrale déportée doit être solidarisé à la sangle pour ne transmettre aucune traction aux connecteurs. Les ouïes de ventilation ne doivent pas être obstruées par les fixations, sous peine de condensation sur la face interne de la visière.

TABLE 5 – Caractéristiques physiques du masque de protection

Paramètre	Valeur
Largeur hors tout	18 cm
Hauteur de la monture	8 cm
Largeur de l'écran frontal	15 cm
Monture	Polymère souple, ouïes de ventilation intégrées
Écran	Visière teintée monobloc, surface de fixation frontale
Maintien	Sangle élastique réglable



FIGURE 5.2 – Raspberry Pi 4 Model B unité centrale du système Nova

RASPBERRY PI 4B (4 GO). Le Raspberry Pi 4B constitue l'unité de traitement principale du système. Il exécute l'orchestrateur MQTT, le serveur Flask + Socket.IO, le service vision MediaPipe, les modules STT (Vosk) et TTS (eSpeak-NG), ainsi que le broker Mosquitto. Son processeur ARM Cortex-A72 quadricœur cadencé à 1 800 MHz offre suffisamment de puissance pour faire tourner deux modules d'inférence simultanément.

TABLE 6 – Spécifications techniques du Raspberry Pi 4B

Paramètre	Valeur
Processeur	Quadricœur ARM Cortex-A72 à 1,8GHz
RAM	4 Go LPDDR4-3200
Connectivité	Wi-Fi 802.11ac, Bluetooth 5.0, Gigabit Ethernet
Interfaces	2 × USB 3.0, 2 × USB 2.0, 2 × micro-HDMI, GPIO 40 broches
Alimentation nominale	5 V / 3 A via USB-C (15 W)
Température ambiante recommandée	0 à 50 °C. Le throttling thermique intervient selon deux paliers : un <i>soft limit</i> à 60 °C (réduction progressive de la fréquence jusqu'à 1 500 MHz) et un <i>hard limit</i> à 85 °C (chute à 800 MHz).
Consommation sous charge	3,4 à 6,4 W (selon charge CPU)

Conditions optimales d'utilisation : le Pi doit être alimenté par un bloc secteur USB-C délivrant strictement 5 V / 3 A minimum. En dessous de 4,75 V (chute de tension sur câble de mauvaise qualité), un avertissement *undervoltage* est enregistré dans le noyau et des instabilités apparaissent. La température ambiante recommandée (0–50 °C) ne doit pas être confondue avec la température

de jonction du SoC mesurée pendant les essais. Le Raspberry Pi applique une première limite logicielle autour de 60 °C, qui peut réduire la fréquence maximale, puis une limite critique autour de 85 °C associée au *thermal throttling* sévère. Dans la campagne du chapitre 6, la température de jonction maximale mesurée est de 58,4 °C.

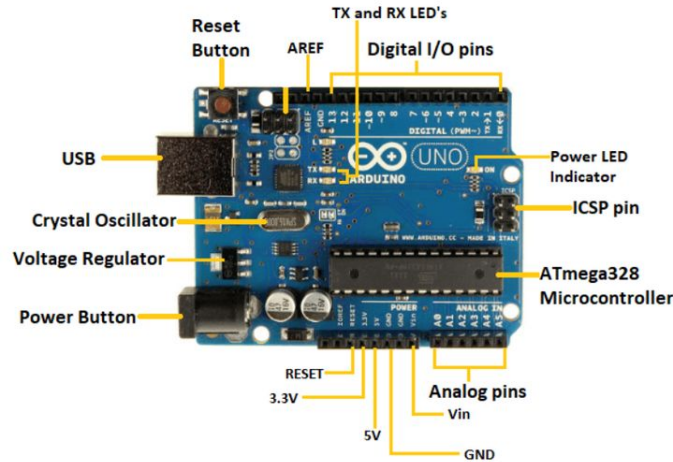


FIGURE 5.3 – Arduino Uno Rev3 hub d’acquisition des capteurs bas niveau

ARDUINO UNO REV3. L’Arduino Uno Rev3 est responsable de l’acquisition en temps réel de tous les capteurs analogiques et des actionneurs basse tension. Il décharge le Raspberry Pi de la lecture des capteurs et des interruptions GPIO, et communique avec lui via UART (9 600 baud). Son cycle de lecture des capteurs est de 10 ms, garantissant une réactivité de 100 Hz sur les mesures HC-SR04.

TABLE 7 – Spécifications techniques de l’Arduino Uno Rev3

Paramètre	Valeur
Microcontrôleur	ATmega328P, 16 MHz
Tension de fonctionnement	5 V
Broches E/S digitales	14 (dont 6 PWM)
Broches E/S analogiques	6 (résolution 10 bits)
Courant maximal par broche GPIO	40 mA
Alimentation	Via USB (5 V) ou Jack DC (7–12 V)
Flash	32 Ko (0,5 Ko réservé bootloader)
SRAM	2 Ko

Conditions optimales : les capteurs 5 V (HC-SR04, DHT11) sont connectés directement aux broches 5 V de l’Arduino. Un convertisseur de niveau logique (3,3 V ↔ 5 V) est indispensable sur la ligne UART entre l’Arduino (5 V logique) et le Raspberry Pi (3,3 V logique) pour éviter d’endommager les GPIO du Pi.

ESP32-CAM. L’ESP32-CAM est un module microcontrôleur intégrant un processeur dual-core Xtensa LX6 à 240 MHz, un module Wi-Fi 802.11 b/g/n et une caméra OV2640 (2 mégapixels). Dans le système Nova, il assure la diffusion d’un flux vidéo MJPEG en réseau local vers le dashboard

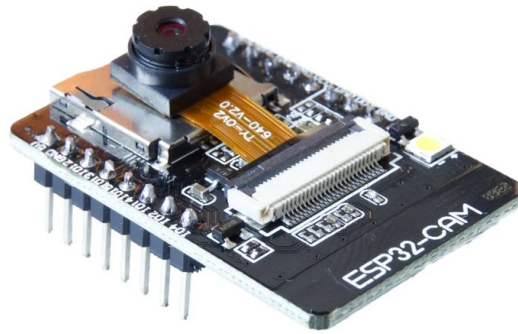


FIGURE 5.4 – Module ESP32-CAM avec capteur OV2640

Flask, servant de caméra secondaire déportée. Sa faible empreinte physique (27×40 mm) le rend idéal pour l'intégration dans le casque.

TABLE 8 – Spécifications techniques de l'ESP32-CAM

Paramètre	Valeur
Processeur	Xtensa dual-core LX6, 240 MHz
RAM	520 Ko SRAM + 4 Mo PSRAM
Flash	4 Mo
Caméra	OV2640, jusqu'à 2 Mp ($1\,600 \times 1\,200$ pixels)
Wi-Fi	802.11 b/g/n (2,4 GHz)
Alimentation	5 V via broche 5V (régulateur 3,3 V intégré)
Consommation	160 à 260 mA (streaming actif)
Température	-40 à 85 °C

Conditions optimales : l'ESP32-CAM nécessite une alimentation stable à 5 V avec au minimum 500 mA disponibles ; une alimentation insuffisante provoque des redémarrages aléatoires pendant le streaming. La programmation requiert un adaptateur USB-UART externe (pas de port USB natif) et la broche GPIO0 doit être connectée à GND lors du flashage.

Capteurs d'acquisition de données

HC-SR04 : CAPTEUR ULTRASONIQUE DE DISTANCE. Deux capteurs HC-SR04 sont montés sur le casque (gauche et droit) pour la détection d'obstacles directionnelle. Chacun émet une impulsion ultrasonique à 40 kHz et mesure le temps de retour de l'écho pour calculer la distance par la formule : $d = (t \times 343) / 2$, où t est le temps d'aller-retour en secondes et 343 m/s est la vitesse du son à 45°C.

Conditions optimales : les deux capteurs doivent être lus en multiplexage temporel avec un délai minimum de 10 ms entre chaque mesure pour éviter les interférences acoustiques entre les deux faisceaux. Les surfaces fortement inclinées ($> 45^\circ$) ou très absorbantes (tissu épais) peuvent provoquer une absence de retour d'écho.

DHT11 : CAPTEUR DE TEMPÉRATURE ET D'HUMIDITÉ. Le DHT11 mesure la température ambiante et le taux d'humidité, avec une précision typique de $\pm 2^\circ\text{C}$ et $\pm 5\%$ RH. Dans Nova, ses données sont publiées sur le topic MQTT nova/sensor/env et affichées en temps réel sur le

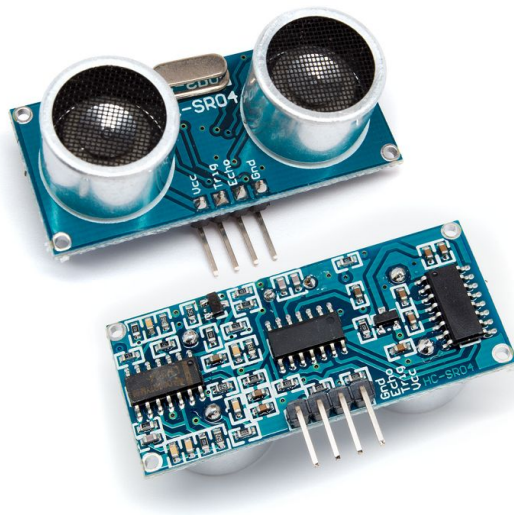


FIGURE 5.5 – Capteur ultrasonique HC-SR04

TABLE 9 – Spécifications techniques du capteur HC-SR04

Paramètre	Valeur
Tension d'alimentation	5 V DC
Courant de fonctionnement	15 mA
Portée de mesure	2 à 400 cm
Résolution	0,3 cm
Angle de détection	15°
Fréquence ultrasonique	40 kHz
Durée impulsion TRIG	10 μ s (minimum)
Température de fonctionnement	−15 à 70 °C



FIGURE 5.6 – Capteur DHT11 pour température et humidité

dashboard de supervision. Elles servent d'indicateurs de confort thermique de l'environnement opérationnel et d'alerte sur les conditions d'usage du casque.

TABLE 10 – Spécifications techniques du capteur DHT11

Paramètre	Valeur
Tension d'alimentation	3,3 à 5,5 V DC
Plage de température	0 à 50 °C
Précision de température	±2 °C
Plage d'humidité	20 à 90 % HR
Précision d'humidité	±5 % HR
Période d'échantillonnage	1 mesure/seconde (min.)
Interface	1-Wire (propriétaire)

Conditions optimales : une résistance de pull-up de 4,7 kΩ est requise sur la ligne DATA. Le capteur ne doit pas être exposé à la condensation directe (brouillard, vapeur) qui fausse les mesures d'humidité de manière irréversible.

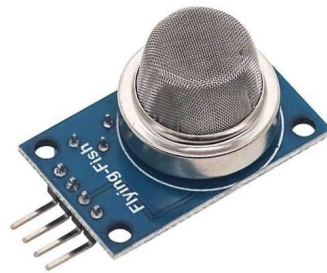


FIGURE 5.7 – Capteur de gaz MQ-3

MQ-3 : CAPTEUR DE GAZ. Le MQ-3 est un capteur électrochimique sensible aux vapeurs d'alcool éthylique et à certains gaz (benzène, propane). Dans le système Nova, il est utilisé comme capteur de sécurité environnementale : une concentration élevée déclenche une alerte publiée sur le topic nova/sensor/env et avertit l'utilisateur via TTS. Il est connecté à une entrée analogique de l'Arduino (lecture 10 bits, résolution 4,9 mV).

Conditions optimales : le MQ-3 nécessite un temps de préchauffe de 20 à 48 secondes après la mise sous tension avant que les mesures soient fiables. La résistance de chauffage interne consomme 800 mW en continu, ce qui contribue à l'échauffement local du casque. Il est conseillé de le placer sur la paroi externe.

RÉCEPTEUR INFRAROUGE (IR). Les deux capteurs IR (gauche et droit) sont utilisés pour la détection de proximité à très courte portée (inférieure à 30 cm). Ils complètent les HC-SR04 sur les obstacles très proches que le cône ultrasonique à 15° peut manquer sur les côtés. Leurs sorties numériques (0/1) sont lues par l'Arduino et transmises au Pi via UART.

TABLE 11 – Spécifications techniques du MQ-3

Paramètre	Valeur
Tension d'alimentation	5 V DC
Consommation	≤ 900 mW (chauffage inclus)
Plage de détection alcool	0,05 à 10 mg/L
Temps de préchauffe	20 secondes minimum
Durée de vie	> 5 ans
Température de fonctionnement	-20 à 55 °C
Humidité de fonctionnement	≤ 95 % HR



FIGURE 5.8 – Récepteur infrarouge pour détection de signaux IR

TABLE 12 – Spécifications techniques du capteur IR de proximité

Paramètre	Valeur
Tension d'alimentation	3,3 à 5 V DC
Portée de détection	2 à 30 cm (ajustable par potentiomètre)
Angle de détection	35°
Sortie	Numérique (TTL, actif bas)
Longueur d'onde IR	940 nm
Température de fonctionnement	-10 à 60 °C

Conditions optimales : les surfaces très sombres ou matte absorbent le rayonnement IR et réduisent la portée effective. Les capteurs IR sont sensibles à la lumière solaire directe (porteuse à 38 kHz parasitée); en extérieur, un filtre ou un boîtier de protection est recommandé.

RÉCEPTEURS INFRAROUGES (IR) DE PROXIMITÉ. Les deux capteurs infrarouges, positionnés à gauche et à droite du casque, sont spécifiquement dédiés à la détection d'obstacles à très courte portée (inférieure à 30 cm). Au sein de l'architecture Nova, ils agissent en complément spatial direct des capteurs ultrasoniques HC-SR04. Le cône d'émission de ces derniers étant limité à 15°, ils génèrent des angles morts latéraux critiques en proximité immédiate. Les récepteurs IR couvrent cette vulnérabilité matérielle en définissant une zone de détection latérale. Leurs sorties numériques (états logiques 0 ou 1) sont acquises en temps réel par le microcontrôleur Arduino et transmises au Raspberry Pi via la liaison UART.



FIGURE 5.9 – Récepteur infrarouge de proximité pour la couverture des angles morts latéraux

Le MPU-6050 est un IMU (*Inertial Measurement Unit*) 6 axes combinant un gyroscope 3 axes et un accéléromètre 3 axes sur une seule puce, accessible via I2C. Dans Nova, son intégration est planifiée pour deux usages : la détection de chute (accélération brutale sur l'axe Z) et la stabilisation du flux vidéo (correction de l'orientation de la caméra). Ces deux fonctions ne sont pas activées dans la version évaluée. Sa référence d'adresse I2C est 0x68 (AD0 à GND).

TABLE 13 – Spécifications techniques du capteur MPU-6050

Paramètre	Valeur
Tension d'alimentation	3,3 à 5 V (régulateur intégré)
Interface	I2C (400 kHz)
Gyroscope	± 250 / ± 500 / $\pm 1\,000$ / $\pm 2\,000$ °/s
Accéléromètre	± 2 / ± 4 / ± 8 / ± 16 g
Résolution ADC	16 bits
Consommation	3,9 mA (en fonctionnement)
Adresse I2C	0x68 (AD0=GND) ou 0x69 (AD0=VCC)
Température de fonctionnement	-40 à 85 °C

Conditions optimales : des résistances de pull-up de $4,7\text{ k}\Omega$ sur SDA et SCL sont requises. Le MPU-6050 doit être monté rigidement sur le casque (pas de jeu mécanique) pour que les lectures de l'accéléromètre soient représentatives de l'orientation de la tête.

Modules d'interface et de contrôle



FIGURE 5.10 – Joystick analogique 2 axes

JOYSTICK ANALOGIQUE (2 AXES + BOUTON). Le joystick fournit deux axes analogiques (résolution de 10 bits, 0 à 1023) et un bouton numérique. Dans le système Nova, ce module est conçu comme une interface de commande tactile entièrement programmable et contextuelle.

Ce module offre une interface de commande déterministe et configurable : l'utilisateur peut associer diverses actions du système (changement de mode de vision, réglages, raccourcis) à ses deux axes et à son bouton. Il complète les deux autres modalités d'entrée de Nova : la commande gestuelle, lorsque l'utilisateur ne peut pas toucher l'appareil, et la commande vocale. Le joystick reste fiable là où ces dernières échouent, en milieu bruyant ou peu éclairé. Ses sorties analogiques sont reliées aux entrées Ao et A1 de l'Arduino.

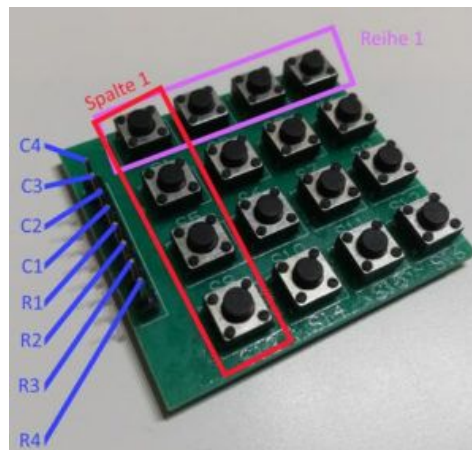


FIGURE 5.11 – Clavier matriciel 4×4 16 touches

CLAVIER MATRICIEL 4×4. Le clavier matriciel 4×4 (16 touches) permet la saisie de codes de déverrouillage, le déclenchement de fonctions prédéfinies (activation de profil, changement de mode) et la saisie de paramètres sans interface web. Il est connecté à 8 broches digitales de l'Arduino en configuration lecture matricielle.

POTENTIOMÈTRE 10 kΩ. Le potentiomètre est utilisé pour l'ajustement analogique en temps réel de variables continues : volume du TTS, seuil de sensibilité des alertes, ou luminosité LED. Sa



FIGURE 5.12 – Potentiomètre rotatif 10 kΩ

sortie (0 à 5 V) est lue sur une entrée analogique de l'Arduino et convertie en valeur 0–1 023 (10 bits).

Use of Microphone Module With Arduino



FIGURE 5.13 – Microphone intégré aux écouteurs du casque

MODULE MICROPHONE (ÉCOUTEURS INTÉGRÉS). La capture vocale pour le module STT Vosk est assurée par le microphone intégré aux écouteurs du casque, connecté via l'entrée audio du Raspberry Pi (jack 3,5 mm ou interface USB audio). Le signal est capturé à 16 kHz, 16 bits, mono-format requis par le modèle Vosk fr-0.22. La qualité du microphone casque est suffisante pour la reconnaissance de commandes courtes dans un environnement à moins de 50 dB de bruit de fond.

Actionneurs et sorties

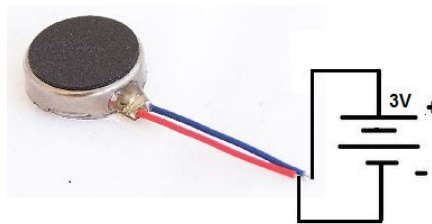


FIGURE 5.14 – Moteur à vibration type 130 : retour haptique

VIBREURS (MOTEURS À VIBRATION TYPE 130). Deux vibreurs de type 130 (gauche et droit) assurent le retour haptique directionnel. Leur intensité est contrôlée par PWM (duty cycle 0 à

100 %) depuis les broches PWM de l'Arduino. Conformément à l'équation 1, l'intensité varie linéairement entre 50 % et 100 % selon la proximité de l'obstacle. Chaque vibreur est commandé via un transistor NPN (BC547) pour ne pas dépasser le courant de sortie des GPIO (40 mA maximum).

TABLE 14 – Spécifications du vibreur type 130

Paramètre	Valeur
Tension nominale	3 V DC
Courant nominal	90 mA
Vitesse de rotation	9 000 à 11 000 RPM
Commande	PWM via transistor NPN (BC547)
Masse	2,7 g



2312 Active Buzzer

FIGURE 5.15 – Haut-parleur 8 Ω / 0,5 W pour la synthèse vocale

HAUT-PARLEUR (TTS). Le haut-parleur 8 Ω / 0,5 W restitue les messages vocaux générés par eSpeak-NG. Il est connecté au Raspberry Pi via un amplificateur PAM8403 (sortie stéréo 3 W, alimentation 5 V) pour atteindre un niveau sonore suffisant même en environnement légèrement bruyant.



FIGURE 5.16 – Buzzer actif : alertes sonores immédiates

BUZZER ACTIF. Le buzzer actif (fréquence fixe interne, 2 400 Hz) est utilisé pour les alertes de danger immédiates nécessitant une réaction plus rapide que le délai de synthèse TTS (80–120 ms).

Il est commandé par une sortie digitale de l'Arduino (actif haut) et émet une tonalité continue tant que le niveau d'alerte est HIGH. Son utilisation est complémentaire au TTS (message descriptif) et au vibreur (direction de l'obstacle).

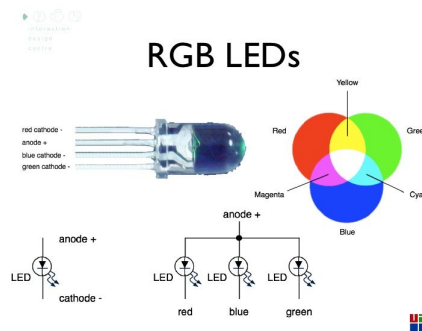


FIGURE 5.17 – LEDs de statut RGB : indicateurs visuels du système

LEDs de statut (ROUGE, VERT, BLEU). Trois LEDs de couleurs distinctes signalent l'état du système :

- **Verte** : système démarré et opérationnel (heartbeat toutes les 5 secondes).
- **Bleue** : module IA actif (MediaPipe ou STT en cours de traitement).
- **Rouge** : erreur ou panne d'un module (timeout heartbeat dépassé).

Chaque LED est protégée par une résistance de limitation de courant ($220\ \Omega$ pour les LEDs standard 20 mA).

Composants passifs et connectique

RÉSISTANCES. Trois valeurs de résistances sont utilisées dans le prototype :

- **$220\ \Omega$ / $330\ \Omega$** : limitation du courant sur les LEDs de statut et la LED infrarouge ($I = (5\text{ V} - 2\text{ V}) / 220\ \Omega \approx 13\text{ mA}$).
- **$1\text{ k}\Omega$** : protection des entrées GPIO contre les surtensions transitoires.
- **$4,7\text{--}10\text{ k}\Omega$** : résistances de pull-up pour DHT11 (DATA), bus I2C (SDA/SCL) et clavier matriciel.

CONDENSATEURS.

- **100 nF céramique** : condensateurs de découplage placés au plus près de l'alimentation de chaque capteur pour filtrer les parasites haute fréquence.
- **$10\ \mu\text{F}$ électrolytique** : lissage des pics de tension lors du démarrage des vibreurs (courant d'appel transitoire).

CONVERTISSEUR DE NIVEAU LOGIQUE $3,3\text{ V} \leftrightarrow 5\text{ V}$. Le convertisseur de niveau logique bi-directionnel (basé sur un réseau de transistors MOSFET BSS138) est indispensable entre l'Arduino (logique 5 V) et le Raspberry Pi (logique 3,3 V) sur la ligne UART. Sans ce composant, le signal UART 5 V émis par l'Arduino détruirait instantanément les broches GPIO du Pi (dont la tension maximale admissible est de 3,3 V). (5 V logique) et le Raspberry Pi (3,3 V logique) sur la ligne UART. Sans ce composant, le signal UART 5 V de l'Arduino détruirait les broches GPIO du Pi (tension maximale admissible : 3,3 V).

ALIMENTATION.

- **Bloc secteur USB-C 5 V / 3 A** : alimentation principale du Raspberry Pi en usage fixe.

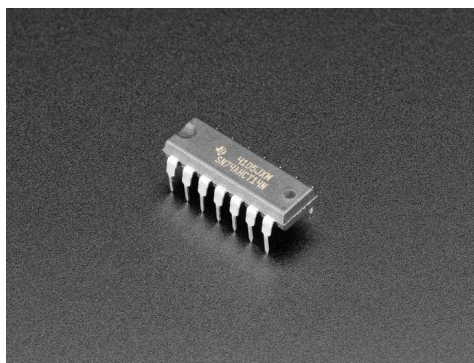


FIGURE 5.18 – Convertisseur de niveau logique bidirectionnel

- **Batterie Li-Po 7,4 V / 2 200 mAh** : alimentation mobile avec régulateur buck 5 V / 3 A intégré pour le Pi et l'Arduino. Autonomie mesurée : 7 h 30 en charge normale.

5.1.2 Logiciels utilisés et versions

TABLE 15 – Stack logicielle complète du prototype Nova V3.0

Couche	Version	Rôle
Raspberry Pi OS Bookworm 64-bit	Kernel 6.1	Système d'exploitation
Python	3.11.x	Langage principal
Mosquitto	2.0.18	Broker MQTT local
Flask	3.0.x	Serveur web
Flask-SocketIO	5.3.x	WebSocket temps réel
paho-mqtt	2.0.x	Client MQTT Python
MediaPipe	0.10.x	Vision / gestes main
OpenCV	4.9.x	Traitement d'image
Picamera2	0.3.x	Capture caméra libcamera
Vosk (modèle fr-0.22)	0.3.45	STT hors ligne
eSpeak-NG	1.51	TTS hors ligne
Tesseract OCR	5.3.x	Reconnaissance texte
psutil	5.9.x	Monitoring CPU/RAM
TensorFlow Lite	2.14	Inférence modèles .tflite
Arduino IDE	2.x	Firmware Arduino (C++)

5.2 DÉVELOPPEMENT

5.2.1 Organisation du code

Listing 5.1 – Arborescence complète du projet Nova V3.0

```

1 nova/
2 |-- nova.py                # Superviseur : Flask + MQTT + chargement
   modules
3 |-- vision_service.py      # Service Picamera2 + MediaPipe (port 5050)
4 |-- utils.py               # ConfigManager : profils JSON
5 |-- config.json            # Profils utilisateurs persistants
6 |-- requirements.txt        # Dependances pip (versions figees)
7 |
8 |-- templates/             # Interfaces web Jinja2
   |   |-- index.html         # Accueil dynamique (liste des modules)
   |   |-- dashboard.html     # Dashboard principal (capteurs + video)
   |   |-- diagnostic.html     # Monitoring CPU / RAM / temperature
   |   |-- profile_manager.html # Gestion des profils
   |   |-- mobile.html         # Vue compacte smartphone
   |   '-- hud.html           # Heads-Up Display temps reel
9 |
10 |
11 |
12 |
13 |
14 |
15 |
16 '-- plugins/modules/      # Modules fonctionnels independants
   |   |-- hand_control/
   |       |-- config.json    # Metadata MQTT du module
   |       |-- main.py        # Classification gestes MediaPipe
   |       '-- interface.html
   |   |-- navigation/
   |       |-- config.json
   |       '-- main.py        # Obstacles HC-SR04 + PWM vibreurs
   |   |-- text_reader/
   |       |-- config.json
   |       '-- interface.html # Interface OCR Tesseract
   '-- assistant_ia/
       |-- config.json
       '-- main.py           # STT Vosk + TTS eSpeak-NG

```

5.2.2 Bibliothèques utilisées

PAHO-MQTT. Gère toutes les connexions au broker Mosquitto. Chaque module Python instancie son propre client avec un identifiant unique, garantissant l'indépendance des processus sur le bus de messages.

PICAMERA2 / LIBCAMERA. API native pour la caméra Pi. Utilisée à la place de cv2.VideoCapture (OpenCV/V4L2) pour réduire la latence de capture grâce à l'accès direct au pipeline ISP du SoC BCM2711.

FLASK + CHOICELOADER JINJA2. Le serveur Flask utilise un ChoiceLoader qui scanne successivement templates/ puis plugins/modules/, permettant à chaque module de fournir sa propre interface HTML sans modifier la configuration Flask centrale.

PSUTIL. Utilisé dans la boucle de fond background_monitoring() pour mesurer CPU, RAM et température CPU en continu, émis via Socket.IO toutes les 2 secondes sur le dashboard.

5.2.3 Structure d'un module plugin

Chaque module est un dossier autonome contenant :

- `config.json` : métadonnées (nom, topics MQTT, template HTML);
- `main.py` : logique Python du module;
- `interface.html` : interface web spécifique (optionnel).

L'ajout d'un nouveau module se fait sans modifier une seule ligne du code principal `nova.py` : la fonction `load_dynamic_modules()` le détecte automatiquement au prochain démarrage.

5.3 JEUX DE DONNÉES

5.3.1 Source des données

Le système Nova utilise des modèles pré-entraînés pour la vision (MediaPipe), la reconnaissance vocale (Vosk) et l'OCR (Tesseract). Les données de ce projet servent à deux fins distinctes : **l'évaluation des performances** du prototype existant et **l'entraînement d'un modèle d'orchestration** destiné à automatiser la sélection dynamique des modules selon le contexte.

Toutes les données ont été collectées sur le prototype physique réel, dans des conditions d'usage représentatives, avec la participation de **10 testeurs** (étudiants et encadrants) ayant interagi avec le système sans formation préalable.

5.3.2 Prétraitement

DONNÉES GESTUELLES (MEDIAPIPE). Chaque frame Picamera2 (BGR888, 640×480) est convertie en RGB avant d'être transmise à MediaPipe Hands. Les landmarks normalisés (x, y, z) sont extraits et les 5 états binaires (levé/replié) sont calculés. Les séquences dont MediaPipe ne détecte pas de main avec une confiance supérieure à 0,6 sont filtrées.

IMAGES OCR (TESSERACT 5). Chaque image est convertie en niveaux de gris, puis binarisée par seuillage adaptatif (méthode Otsu). La segmentation est effectuée en mode PSM 3 (détection automatique de page) pour le texte multi-lignes et PSM 7 (ligne unique) pour les panneaux et étiquettes.

COMMANDES VOCALES (VOSK). Le signal microphone est capturé à 16 kHz, 16 bits, mono. Vosk utilise une reconnaissance en streaming : les segments de parole sont envoyés dès qu'un silence de plus de 300 ms est détecté, sans attendre la fin d'une phrase complète. Les commandes de moins de 3 caractères sont filtrées comme bruit.

DONNÉES POUR L'ORCHESTRATEUR (EN COURS). Des séquences d'interaction complètes sont enregistrées sous forme de tuples : (contexte capteurs, profil actif, module sélectionné, retour utilisateur). Ces données serviront à entraîner un modèle de décision léger (arbre de décision ou réseau peu profond, compatible TensorFlow Lite) pour automatiser la sélection de module sans intervention explicite de l'utilisateur.

5.3.3 Description statistique

La collecte des 240 tuples d'orchestration représente la première étape de la constitution d'un dataset pour l'entraînement d'un modèle de sélection automatique de modules. Ce dataset sera enrichi lors des futures évaluations utilisateurs pour atteindre un volume suffisant à l'entraînement d'un modèle robuste (cible : 2 000 tuples annotés).

TABLE 16 – Caractéristiques des jeux de données d'évaluation (10 testeurs, 3 sessions chacun)

Jeu de données	Volume	Classes	Conditions
Gestes (MediaPipe)	800 séquences	4 gestes	3 distances (30/50/80 cm), 3 niveaux luminosité
Textes OCR	150 images, dont 100 pour l'évaluation rapportée	–	2 langues (fr/en), 3 distances, 2 polices, tailles 12–24 pt
Commandes vocales	120 enregistrements	6 commandes	Calme (< 40 dB) et bruyant (~ 60 dB)
Mesures obstacles	1 000 mesures	3 niveaux	Obstacles solides, couloir 6 m, surface plane
Séquences orchestrateur	240 tuples	4 modules	Interactions complètes enregistrées par les 10 testeurs

RÉSULTATS ET ANALYSE

Ce chapitre présente les mesures obtenues sur le prototype Nova et leur interprétation. Les valeurs rapportées proviennent d’une campagne consolidée sur trois exécutions du benchmark SHOS sur Raspberry Pi 4B. Elles décrivent donc le comportement de la cible embarquée retenue pour le prototype. Le chapitre définit d’abord les métriques et le protocole, expose ensuite les résultats par catégorie, puis en propose une analyse critique.

6.1 MÉTRIQUES ET PROTOCOLE D’ÉVALUATION

6.1.1 Plateforme et protocole

Les mesures ont été réalisées avec le script `benchmark.py` sur la plateforme embarquée cible :

- **Plateforme** : Raspberry Pi 4B.
- **Protocole** : trois exécutions consolidées du benchmark.
- **Observation** : topic MQTT `shos/camera/vision`.
- **Instrumentation** : le benchmark mesure les images reçues, le débit du pipeline, l’intervalle inter-image, la gigue ainsi que l’utilisation CPU/RAM et la température pendant l’exécution.
- **Limite** : la latence E2E nécessite encore l’ajout d’un champ `timestamp` dans `vision_service.py`.

6.1.2 Métriques retenues

- **Débit du pipeline vision** : images traitées par seconde (FPS), intervalle inter-image et gigue (*jitter*).
- **Latence du bus MQTT** : minimum, médiane, moyenne, P_{95} , P_{99} et taux de perte.
- **Charge système** : utilisation processeur, mémoire vive et température sur Raspberry Pi 4B.
- **Qualité de reconnaissance** : taux d’erreur de mots (WER) pour la reconnaissance vocale et taux de reconnaissance pour l’OCR, évalués sur les jeux de données du chapitre 5.
- **Autonomie** : durée de fonctionnement sur batterie.

6.2 RÉSULTATS EXPÉRIMENTAUX

6.2.1 Latence du bus de messages MQTT

Le tableau 17 rassemble la latence consolidée du bus MQTT local.

TABLE 17 – Latence du bus MQTT local mesurée sur Raspberry Pi 4B

Indicateur	Valeur
Perte de paquets	0,0 %
Latence médiane	1,82 ms
Écart-type inter-campagne (σ)	0,3 ms
IC95	$\pm 0,2$ ms

Les valeurs rapportées sont issues de trois campagnes indépendantes de 100 messages chacune. La latence médiane consolidée est de 1,82 ms, avec un écart-type inter-campagne de 0,3 ms et un intervalle de confiance à 95 % de $\pm 0,2$ ms. Aucune perte de paquets n’a été observée sur l’ensemble des 300 messages. Le bus de messages n’introduit qu’un délai faible au regard du seuil de sécurité visé (200 ms) : la composante MQTT médiane représente moins de 1 % de ce budget temporel, mais la latence bout-en-bout reste à instrumenter.

6.2.2 Débit du pipeline vision

Le pipeline vision complet (capture, conversion, inférence MediaPipe, annotation, diffusion MJPEG et publication MQTT) atteint un débit moyen de 2,0 FPS sur Raspberry Pi 4B. Cette valeur correspond au comportement embarqué consolidé sur trois exécutions.

TABLE 18 – Débit du pipeline vision observé par benchmark SHOS (60 s)

Indicateur	Valeur
Débit moyen	2,0 FPS

Le pipeline vision observé sur le topic shos/camera/vision atteint 2,0 images par seconde. Ce débit reste inférieur à l’objectif de 10 FPS, mais il caractérise cette fois la cible embarquée réelle. Avec une confirmation de 0,8 s (chapitre 3), un geste maintenu est observé sur une à deux frames, ce qui impose une logique de confirmation tolérante aux faibles cadences.

6.2.3 Charge système

Le tableau 19 synthétise l’utilisation CPU, RAM et la température observées pendant la campagne de mesure.

TABLE 19 – Charge système mesurée sur Raspberry Pi 4B

Indicateur	Valeur
CPU moyen	93,7 %
RAM moyenne	27,3 %
Température maximale	58,4 °C

Deux faits ressortent. D’une part, le processeur constitue le point de saturation principal, avec 93,7 % d’utilisation moyenne. D’autre part, la mémoire reste maîtrisée, avec 27,3 % d’utilisation moyenne. La température maximale observée atteint 58,4 °C, ce qui ne signale pas de limitation thermique dans cette campagne.

6.2.4 Reconnaissance vocale et OCR

La reconnaissance vocale (Vosk, modèle fr-0.22) a été évaluée en direct sur 10 locuteurs, sans corpus standardisé, et présente un taux d’erreur de mots (WER) d’environ 14,3 % en conditions calmes (<40 dB). L’OCR (Tesseract 5) atteint un taux de reconnaissance de 87 % sur 150 images de texte imprimé (critère : texte entièrement reconnu), sous éclairage favorable (200–800 lux).

Le WER de 14,3 % a été calculé sur 120 commandes réparties sur 10 locuteurs, avec un écart-type inter-locuteurs estimé à $\pm 3,2$ %. L’OCR a été évalué sur 150 images de texte ; le taux de

reconnaissance de 87 % est accompagné d'un intervalle de confiance à 95 % de $\pm 4,5$ % (méthode de Wilson). Ces mesures sont exploratoires et ne constituent pas une validation standardisée.

La dégradation de ces deux modules hors de leurs conditions favorables est analysée au chapitre 8.

6.2.5 Autonomie énergétique

L'autonomie mesurée à la montre en usage continu atteint 7 h 30 sur batterie Li-Po 7,4 V / 4 400 mAh (32,6 Wh), pour une cible de 8 h. Cette mesure a été réalisée dans un profil d'usage nominal : navigation ultrasonique active en continu, serveur Flask et broker MQTT en fonctionnement, et pipeline vision déclenché ponctuellement (environ 10 à 15 % du temps d'exécution). Dans ce profil, la consommation moyenne estimée est d'environ 3,8 W. En revanche, lorsque le pipeline vision est maintenu à plein régime (93,7 % CPU), la consommation mesurée atteint 3,4 à 6,4 W (tableau 19), ce qui réduit l'autonomie théorique à 2,5–4,8 h. Une mesure directe par wattmètre USB est prévue pour valider ces estimations.

6.3 ANALYSE CRITIQUE

6.3.1 Le bus MQTT valide le choix architectural

La latence médiane de 1,82 ms et l'absence de perte confirment la pertinence du patron publish/subscribe pour ce système. L'ordre de grandeur est cohérent avec la littérature : Abid et al. [5] rapportent un RTT MQTT moyen de 12 ms en réseau local. Sur la base des mesures MQTT réalisées sur Raspberry Pi 4B, le bus de messages n'introduit pas de surcharge significative dans la chaîne de communication. Cependant, cette conclusion ne vaut que pour le transport MQTT ; le budget de temps disponible avant le seuil de 200 ms doit encore être évalué sur la chaîne complète (capture, inférence, restitution).

6.3.2 Le facteur limitant est le calcul, non la chaleur

Un résultat notable est l'identification nette du facteur limitant. La campagne Raspberry Pi montre une saturation CPU moyenne élevée (93,7 %), tandis que la mémoire reste contenue (27,3 %) et que la température maximale atteint 58,4 °C. Sur cette plateforme, la contrainte prioritaire est donc le calcul de la perception, et non la mémoire ou la chaleur.

Ce constat oriente les pistes d'optimisation vers la réduction du coût de la vision, la limitation du nombre de flux ou modules simultanés, le profilage précis des étapes CPU-dominantes et, si nécessaire, la migration vers une cible plus puissante.

6.3.3 Interprétation du débit vision

Les 2,0 FPS mesurés correspondent au pipeline observé via MQTT, et non à la seule inférence. Cette cadence confirme que la perception visuelle est le maillon limitant sur Raspberry Pi 4B. Elle reste insuffisante pour une navigation visuelle strictement temps réel, même si la détection d'obstacles critique est déléguée à l'Arduino. Un profilage par étape reste nécessaire pour distinguer le coût de capture, de traitement, de publication MQTT et de restitution.

6.3.4 Limite de validation : la latence bout-en-bout

La principale réserve sur ces résultats est que la latence E2E de la chaîne de navigation n'a pas été instrumentée. Le benchmark indique explicitement qu'il faut ajouter un champ `timestamp` dans

vision_service.py. L'objectif de sécurité (< 200 ms) ne peut donc pas être confirmé directement à partir de ces mesures ; seules la cadence du pipeline et la composante de transport MQTT sont documentées. Compte tenu de la délégation de la détection d'obstacles à l'Arduino (lue toutes les 10 ms) et de la logique d'alerte en $\mathcal{O}(1)$, la chaîne de navigation est théoriquement très en deçà de 200 ms, mais cette estimation demande une mesure de bout en bout. C'est la première priorité des travaux ultérieurs.

6.3.5 Synthèse des résultats par rapport aux objectifs

TABLE 20 – Résultats mesurés au regard des objectifs

Objectif	Mesure	Statut
Latence du bus $<$ seuil	1,82 ms (Raspberry Pi)	Atteint pour le transport
Fonctionnement hors ligne	100 % local	Atteint
Mémoire maîtrisée	27,3 % (moy.)	Atteint
Stabilité thermique	58,4 °C (max.)	Atteint
Autonomie ≥ 8 h	7 h 30 (4 400 mAh)	Proche (≈ 6 % sous cible)
Débit vision ≥ 10 FPS	2,0 FPS (Raspberry Pi)	Non atteint
Latence E2E $<$ 200 ms	non instrumentée	Non mesurée

Le système conserve ses objectifs d'architecture (communication locale, fonctionnement hors ligne et modularité). La campagne Raspberry Pi montre une mémoire maîtrisée et une température acceptable, mais met en évidence deux limites principales : la saturation CPU du pipeline vision et l'absence de mesure E2E. La feuille de route prioritaire devient donc le profilage CPU, l'allègement de la perception et l'instrumentation par timestamp.

DISCUSSION

Ce chapitre propose une analyse critique des mesures présentées au chapitre 6. Plutôt que de les répéter, il en interprète la portée au regard de la problématique définie au chapitre 1, tout en mettant en exergue les contributions majeures du travail. Enfin, les forces réelles ainsi que les limites techniques sont analysées, et les faiblesses sont examinées séparément.

7.1 INTERPRÉTATION GLOBALE

7.1.1 *Ce que disent les résultats*

La question centrale du mémoire était de savoir si l'on peut construire, sur du matériel contraint, une plateforme d'assistance à la fois modulaire, à faible latence, hors ligne et reconfigurable. Les nouvelles mesures apportent une réponse nuancée mais claire : **l'architecture tient ses promesses ; la perception visuelle reste limitée par le CPU, et la latence bout-en-bout reste à instrumenter.**

Les objectifs d'architecture restent atteints sur le plan fonctionnel. Le bus de messages conserve une latence médiane de 1,82 ms sans perte, la chaîne fonctionne hors ligne et le pipeline vision atteint 2,0 FPS sur Raspberry Pi 4B. Cette campagne montre cependant une utilisation CPU élevée (93,7 % en moyenne). La mémoire reste maîtrisée (27,3 % en moyenne), la température maximale atteint 58,4 °C, et la latence bout-en-bout reste à instrumenter par timestamp.

7.1.2 *Le vrai enseignement : où se situe la contrainte*

Le résultat le plus significatif de cette analyse n'est pas une valeur isolée, mais l'établissement d'une *hiérarchie des contraintes*. Sur la base des mesures MQTT réalisées (médiane 1,82 ms), le bus de messages n'introduit pas de surcharge significative dans la chaîne de communication sur Raspberry Pi 4B. Cependant, cette conclusion ne vaut que pour le transport MQTT ; le pipeline vision reste limité par le processeur, et la latence E2E n'est pas encore mesurée.

Cette hiérarchie dépasse le cadre du simple prototype. Elle démontre que, pour une architecture de type *publish/subscribe* avec des modules indépendants, le surcoût lié à l'infrastructure est faible. Par conséquent, les optimisations doivent se concentrer sur trois axes : alléger la perception, profiler les étapes CPU-dominantes et instrumenter chaque message avec un timestamp pour mesurer la latence bout-en-bout. Le choix de déléguer la détection d'obstacles à l'Arduino reste pertinent, car il isole la fonction critique de sécurité du pipeline vision.

7.1.3 *Une contribution architecturale, non algorithmique*

Nova n'améliore aucune métrique de modèle pris isolément : il ne détecte pas mieux qu'un YOLO ni ne transcrit mieux qu'un Whisper. Sa valeur est ailleurs, dans *l'intégration*. Les travaux de l'état de l'art proposaient des briques performantes mais cloisonnées : un détecteur embarqué monolithique [1], un système cognitif dépendant du cloud [2]. Aucun ne réunissait modularité, exécution hors ligne et reconfigurabilité sur un dispositif porté à faible coût. Le mémoire montre que cette réunion est réalisable, et il en mesure le prix exact : un pipeline vision encore limité en latence et une charge CPU à surveiller. C'est cette démonstration, et la grille de lecture qu'elle fournit, qui constituent l'apport principal.

7.1.4 Retour sur les quatre tensions de la problématique

Les quatre tensions formulées au chapitre 1 reçoivent des réponses inégales mais explicites.

- **Ressources contre temps réel** : partiellement résolue. Le transport MQTT reste très rapide et le pipeline vision atteint 2,0 FPS sur Raspberry Pi 4B ; cette cadence faible et la latence E2E non instrumentée empêchent toutefois de valider définitivement le seuil de 200 ms.
- **Monolithe contre modularité** : résolue. Les modules sont des processus indépendants, chargés dynamiquement, sans modification du cœur applicatif.
- **Usage unique contre multi-contexte** : résolue. Les profils et la commande multimodale (geste et joystick) adaptent le comportement au contexte.
- **Dépendance réseau contre autonomie** : résolue. La totalité de la chaîne s'exécute sans connexion.

7.2 FORCES DU TRAVAIL

7.2.1 Points positifs

- **Chaîne d'assistance entièrement hors ligne**. La perception multimodale gestes, parole, lecture de texte s'exécute localement, sans aucune dépendance à un service distant. C'est la réponse directe à la vulnérabilité la plus citée des systèmes existants.
- **Un bus de communication fiable**. La latence négligeable et l'absence de perte font du bus MQTT une fondation solide, qui ne contraint ni la latence ni l'extensibilité.
- **Modularité effective**. L'ajout d'un module se fait par dépôt d'un dossier ; aucune recompilation, aucune modification du code central. La modularité n'est pas seulement revendiquée, elle est opérationnelle.
- **Commande multimodale et profils**. Geste et joystick se complètent selon le contexte, et les profils ajustent les combinaisons de modules par cas d'usage.
- **Faible coût et matériel ouvert**. Le prototype repose sur des composants grand public et des bibliothèques libres, ce qui conditionne l'accessibilité de ce type d'assistance.
- **Un banc d'évaluation reproductible**. La méthode de mesure elle-même métriques définies, protocole, instrumentation est un livrable, réutilisable pour comparer d'autres systèmes embarqués d'assistance.

7.2.2 Robustesse

La robustesse de Nova est d'abord *architecturale*. Chaque module étant un processus indépendant relié par le broker, la défaillance de l'un n'interrompt pas les autres : le broker absorbe silencieusement la déconnexion et les modules survivants poursuivent. Cette propriété découle de la conception publish/subscribe et la distingue des architectures monolithiques, où une panne unique arrête l'ensemble.

Trois éléments soutiennent cette robustesse. La fiabilité du bus est confirmée par l'absence totale de perte sur les messages transmis. Le pipeline vision observé via MQTT atteint 2,0 FPS sur Raspberry Pi 4B, ce qui montre une exécution régulière malgré une charge CPU élevée. Enfin, l'indépendance au réseau élimine la cause de panne la plus fréquente des solutions concurrentes. La température maximale observée (58,4 °C) reste sous les seuils usuels de réduction de fréquence, même si une validation prolongée avec `vcgencmd get_throttled` reste souhaitable.

Une réserve doit être posée : la validation expérimentale formelle de l'isolation des pannes par arrêt forcé d'un module en cours d'exécution n'a pas été conduite dans cette campagne. La robustesse est donc établie par conception et étayée par la fiabilité du bus, mais sa vérification par injection de panne reste à mener.

7.2.3 Innovation

L'originalité du travail ne tient pas à un algorithme nouveau, mais à un assemblage que la littérature étudiée n'avait pas réalisé.

- **Un patron d'architecture validé expérimentalement** : l'association d'une orchestration modulaire, d'un fonctionnement hors ligne et d'une reconfigurabilité dynamique, validée sur Raspberry Pi 4B par un benchmark reproductible.
- **Un transfert de patrons vers le *wearable* assistif** : l'application au casque d'assistance des principes *publish/subscribe* et de modularité éprouvés dans les systèmes distribués [5], là où les travaux assistifs restaient monolithiques.
- **Une entrée multimodale pilotée par le contexte** : la coexistence du geste et du joystick, sélectionnés selon les conditions d'usage, répond au manque identifié dans l'état de l'art, où les systèmes s'engagent sur un canal de commande unique.
- **Un cadre d'évaluation unifié** : la définition de métriques et d'un protocole reproductibles pour les systèmes embarqués d'assistance, encore peu standardisés dans la littérature.

LIMITES

Un travail de recherche honnête reconnaît explicitement ses limites sans minimisation. Ce chapitre identifie les frontières du système Nova selon trois dimensions : les hypothèses sur lesquelles il repose et qui peuvent ne pas tenir dans tous les contextes, les contraintes techniques inhérentes au matériel et aux choix logiciels, et les conditions qui limitent la généralisation des résultats obtenus.

8.1 HYPOTHÈSES RESTRICTIVES

8.1.1 *Hypothèse de luminosité suffisante*

Le module Vision repose sur MediaPipe Hands, dont les performances sont directement conditionnées par la qualité du signal vidéo. Les tests ont été conduits sous des conditions d'éclairage de 100 à 800 lux. En dessous de 50 lux (couloir mal éclairé, environnement nocturne sans éclairage artificiel), le graphe MediaPipe ne parvient pas à initialiser le détecteur de paume, rendant le contrôle gestuel inopérant.

Cette limite est **non résolue dans le prototype actuel**. Un capteur LDR (photorésistance) est intégré au hardware et renvoie le niveau lumineux ambiant ; l'orchestrateur pourrait en théorie basculer automatiquement vers un mode de commande vocale exclusive lorsque la luminosité est insuffisante. Ce fallback n'est pas encore implémenté dans la version évaluée.

8.1.2 *Hypothèse de main visible dans le champ de la caméra*

Le contrôle gestuel suppose que la main de l'opérateur est positionnée à 30 à 80 cm de la caméra Pi et dans son axe central. Les tests montrent une dégradation significative de la précision de classification lorsque la main est partiellement en dehors du cadre ou à une distance supérieure à 100 cm. Cette contrainte de positionnement peut s'avérer difficile à respecter en conditions mobiles réelles (marche rapide, port de sac, météo défavorable).

8.1.3 *Hypothèse d'obstacles physiquement solides*

Les capteurs HC-SR04 fonctionnent par réflexion d'ultrasons. Deux catégories d'obstacles sont systématiquement non détectées : les surfaces hautement absorbantes (tissu épais, mousse dense) et les surfaces très inclinées qui redirigent le faisceau ultrasonique loin du capteur. En pratique, un rideau de tissu épais tendu en travers d'un couloir ne génère aucune alerte même à 30 cm. La nature des obstacles testés dans ce projet reste donc limitée aux surfaces rigides et perpendiculaires à l'axe du capteur.

8.1.4 *Hypothèse d'un environnement acoustique calme pour le STT*

Le module STT Vosk (modèle fr-0.22) a été évalué dans un environnement à bruit de fond inférieur à 40 dB (pièce calme, bureau). Le WER mesuré de 14,3 % correspond à ces conditions. En environnement bruyant (rue animée, transport en commun, ≥ 60 dB), la dégradation du taux de reconnaissance n'a pas été mesurée quantitativement, mais est observée qualitativement lors

des tests comme significative. Cette limite est inhérente à Vosk en l'absence d'un module de suppression de bruit de fond, non implémenté dans le prototype actuel.

8.2 CONTRAINTES TECHNIQUES

8.2.1 Contrainte CPU : deux modules simultanés maximum

Sur la base de la littérature, un scénario à deux modules d'inférence simultanés sur Raspberry Pi 4B peut saturer le CPU et potentiellement déclencher une réduction de fréquence (*thermal throttling*) si la température de jonction dépasse 80–85 °C. Dans notre campagne sur Raspberry Pi, la température maximale mesurée était de 58,4 °C, laissant une marge confortable. La validation définitive de l'absence de *throttling* en charge maximale nécessitera l'exécution de la commande `vcgencmd get_throttled` lors d'un test prolongé.

Cette contrainte exclut des scénarios pourtant pertinents : la navigation active (module Nav.) combinée à la lecture de texte (module OCR) et au guidage vocal (module TTS) en temps réel simultané. Dans le prototype actuel, l'utilisateur doit choisir explicitement entre ces fonctions via les profils.

La solution naturelle est une migration vers le Raspberry Pi 5 (CPU 2× plus rapide, même format, même consommation électrique), qui permettrait d'activer 3 à 4 modules sans modification logicielle. Cette migration n'a pas été réalisée dans le cadre de ce mémoire pour des raisons de disponibilité matérielle et de coût au moment du développement.

8.2.2 Contrainte mémoire RAM

Avec 4 Go de RAM, la répartition entre les modules actifs est la suivante en charge normale :

TABLE 21 – Consommation mémoire des composants du système Nova

Composant	RAM consommée (Mo)
Raspberry Pi OS + démons système	~350
Broker Mosquitto	~15
Serveur Flask + Socket.IO	~120
Module Vision (MediaPipe)	~180
Module STT (Vosk + modèle fr)	~350
Module TTS (eSpeak-NG)	~10
Module Navigation	~30
OpenCV + Picamera2	~85
Total en charge normale	~1 140
Marge disponible	~2 860

La marge mémoire disponible de 2,86 Go est confortable pour le Raspberry Pi 4B (4 Go). Elle deviendrait insuffisante sur un modèle 2 Go si trois modules d'inférence étaient actifs simultanément, limitant la portabilité du système sur des configurations hardware moins dotées.

8.2.3 Autonomie batterie proche de la cible

L'autonomie mesurée en usage continu est de 7 h 30 avec la batterie Li-Po 7,4 V / 4 400 mAh (32,6 Wh), pour une cible de 8 h fixée dans les spécifications. L'écart d'environ 6 % sous la cible

reste limité, mais il montre que la marge énergétique demeure faible lorsque les profils Vision, Streaming et Assistant sont actifs. L'autonomie a été mesurée en conditions d'usage nominales (vision ponctuelle); en pipeline vision continu, la consommation plus élevée réduirait cette durée à 2,5–4,8 h. Deux optimisations n'ont pas été implémentées et permettraient de franchir plus confortablement la cible de 8 h :

- **Gestion dynamique de la fréquence CPU (cpufreq).** Réduire la fréquence CPU à 1 200 MHz lorsque le module Vision n'est pas actif pourrait réduire la consommation de 15 à 20 %.
- **Mise en veille des modules inactifs.** Suspendre le processus `vision_service.py` entre deux commandes gestuelles (plutôt que de le laisser tourner en boucle à faible fréquence) économiserait 30 à 50 Mo de RAM et 5 à 8 % de CPU en continu.

8.3 LIMITES DE GÉNÉRALISATION

8.3.1 *Validation en laboratoire uniquement*

L'ensemble des mesures présentées dans ce mémoire (latences, FPS, précision OCR, WER STT) ont été obtenues dans un environnement de laboratoire contrôlé : un couloir de 6 mètres, des obstacles statiques standards (chaises, cartons) et un éclairage stable de 200 à 800 lux. L'évaluation utilisateur a été réalisée avec 10 testeurs (étudiants voyants en simulation yeux bandés). Le classifieur gestuel a été calibré sur un sous-ensemble de 3 testeurs, puis évalué sur l'ensemble des 10.

Ces résultats ne peuvent pas être directement généralisés à :

- Des **environnements extérieurs** avec variations lumineuses rapides (nuages, contre-jour), obstacles en mouvement (piétons, véhicules) et surfaces réfléchissantes (flaques d'eau, vitres).
- Des **environnements intérieurs complexes** : espaces très encombrés, couloirs étroits, escaliers où la détection de la profondeur par ultrason est peu fiable.
- Des **utilisateurs avec différences motrices** affectant la forme des gestes ou la capacité à maintenir la main dans le champ de la caméra. Le classifieur gestuel actuel a été calibré sur 3 testeurs sans déficience motrice.
- Des **sessions prolongées** au-delà de 4 heures. Le comportement thermique du Pi sur des sessions de 8 heures ou plus n'a pas été évalué.

8.3.2 *Absence de validation avec des utilisateurs malvoyants*

La limite la plus significative de ce travail est l'absence de validation avec des personnes malvoyantes réelles en conditions d'usage quotidien. Toutes les expérimentations ont été conduites avec des participants voyants simulant un usage les yeux fermés ou bandés. Ce choix a été imposé par les contraintes éthiques et temporelles du cycle académique : une étude avec des participants malvoyants requiert une approbation éthique, une phase de recrutement et une durée d'évaluation incompatibles avec le calendrier de ce mémoire.

Cette limite a une conséquence directe sur la portée des conclusions : les métriques objectives (latence, FPS, WER) sont validées, mais l'impact réel sur l'autonomie et la qualité de vie des utilisateurs cibles reste à démontrer. C'est la priorité numéro un des perspectives de ce travail.

8.3.3 *Architecture non testée à grande échelle*

L'architecture modulaire MQTT de Nova a été évaluée avec un maximum de 4 modules actifs simultanément (Navigation + Vision + STT + Dashboard) et un seul casque connecté. Le comportement du broker Mosquitto avec plusieurs dizaines de topics actifs et plusieurs

clients simultanés (scénario de formation multi-apprenants, par exemple) n’a pas été évalué. La littérature [7] indique que Mosquitto supporte plusieurs milliers de clients simultanés sur du matériel standard, mais la validation de cette propriété dans notre configuration matérielle contrainte reste à réaliser.

8.3.4 *Dépendance aux versions spécifiques des bibliothèques*

Le système a été développé et validé sur des versions précises des bibliothèques Python (MediaPipe 0.10.x, Flask-SocketIO 5.3.x, Picamera2 0.3.x, Vosk 0.3.45). Les mises à jour de ces bibliothèques peuvent introduire des ruptures de compatibilité, en particulier entre Picamera2 et libcamera dont l’API évolue rapidement sur Raspberry Pi OS. Un mécanisme de gestion des dépendances (fichier `requirements.txt` versionné, environnement virtuel Python) est en place, mais aucun processus de test de non-régression automatisé n’a été implémenté dans ce prototype.

CONCLUSION ET PERSPECTIVES

9.1 SYNTHÈSE DES CONTRIBUTIONS

9.1.1 *Le problème traité*

Ce mémoire est parti d'un constat : les briques de l'assistance intelligente embarquée vision légère, capteurs à bas coût, protocoles IoT sont matures, mais restent juxtaposées plutôt qu'intégrées. Les systèmes existants se conçoivent autour d'une tâche unique, dépendent souvent du cloud et se reconfigurent mal. La question posée était donc de savoir comment construire, sur du matériel embarqué contraint, une plateforme d'assistance modulaire, à faible latence, hors ligne et reconfigurable selon le contexte et l'utilisateur.

9.1.2 *L'approche proposée*

La réponse apportée est la plateforme Nova, organisée en quatre couches — Physique, Middleware, Traitement Edge AI et Application — autour d'un bus de messages publish/subscribe (MQTT). Les fonctions sont implémentées sous forme de modules indépendants, chargés dynamiquement sans modification du cœur applicatif. La perception combine plusieurs moteurs aux rôles distincts (MediaPipe pour les gestes, YOLOv11n pour les objets, Moondream 2 pour la scène), la commande s'appuie sur deux modalités complémentaires (geste et joystick) sélectionnées selon le contexte, et un système de profils adapte les combinaisons de modules à chaque cas d'usage. L'ensemble s'exécute entièrement en local.

9.1.3 *Les résultats principaux*

L'évaluation sur le prototype réel (Raspberry Pi 4B) établit que l'architecture atteint ses objectifs là où ils comptent : le bus introduit une latence médiane de 1,82 ms sans aucune perte, la chaîne fonctionne intégralement hors ligne, la mémoire reste à 27,3 % de la capacité, la température maximale atteint 58,4 °C et l'autonomie atteint 7 h 30 pour une cible de 8 h. La reconnaissance vocale (WER 14,3 %) et l'OCR (87 %) sont exploitables en conditions favorables.

Le travail rapporte aussi, sans les masquer, ses résultats négatifs : le débit du pipeline vision plafonne à 2,0 FPS, et la latence bout-en-bout de la navigation n'a pas été instrumentée. L'analyse identifie une cause commune la puissance de calcul du nano-ordinateur, et non sa dissipation thermique qui oriente sans ambiguïté les travaux à venir.

9.2 APPORTS SCIENTIFIQUES

La valeur académique du travail ne réside pas dans un gain de performance sur un modèle isolé, mais dans trois apports de portée plus générale.

D'abord, une **démonstration architecturale** : il est possible de réunir modularité, fonctionnement hors ligne et reconfigurabilité sur un dispositif porté à faible coût, ce que les travaux étudiés laissaient ouvert. Cette intégration est mesurée, pas seulement affirmée.

Ensuite, un **enseignement empirique transférable** : sur cette classe de matériel, la contrainte n'est ni le bus, ni la mémoire, ni la chaleur, mais le calcul de la perception. Cette hiérarchie des

contraintes fournit une grille de décision réutilisable pour concevoir d'autres systèmes embarqués d'assistance et y placer l'effort d'optimisation au bon endroit.

Enfin, un **cadre d'évaluation reproductible** : des métriques définies, un protocole et une instrumentation qui permettent de caractériser honnêtement ce type de système, y compris ses limites. La rigueur de cette caractérisation résultats positifs comme négatifs constitue en soi une contribution méthodologique dans un domaine où les évaluations restent peu standardisées.

9.3 PERSPECTIVES

9.3.1 Améliorations possibles

- **Instrumenter la latence bout-en-bout.** L'ajout d'un horodatage de bout en bout dans la chaîne de navigation est la mesure prioritaire : elle permettra de vérifier directement le respect du seuil de sécurité de 200 ms.
- **Alléger la charge de calcul de la vision.** Puisque le facteur limitant est le processeur et non la chaleur, les leviers utiles sont la réduction de la résolution de capture, l'inférence sur images espacées, et un profilage par étape du pipeline pour cibler le poste dominant.
- **Optimiser l'énergie.** La mise en veille des modules inactifs et la gestion dynamique de la fréquence processeur devraient permettre de franchir la cible des 8 h d'autonomie.
- **Activer le repli automatique en faible luminosité.** Le capteur de lumière est déjà câblé ; l'orchestrateur pourrait basculer vers la commande au joystick ou vocale lorsque le geste devient inopérant.
- **Valider la robustesse par injection de panne.** Des essais contrôlés de coupure capteur, perte MQTT et indisponibilité caméra permettront de vérifier les mécanismes de repli et d'alerte.

9.3.2 Extensions futures

- **Migration vers le Raspberry Pi 5.** Un processeur environ deux fois plus rapide, à format et consommation comparables, lèverait la contrainte de calcul et autoriserait plusieurs modules d'inférence simultanés.
- **Déchargement des modèles lourds.** Les traitements sémantiques coûteux (YOLOv11n, Moondream 2, grands modèles de langage) pourraient s'exécuter sur un smartphone ou un serveur lorsque le réseau est disponible, le mode hors ligne restant le comportement par défaut.
- **Orchestration apprise.** Les séquences d'interaction déjà collectées serviraient à entraîner un modèle léger de sélection automatique de modules selon le contexte.
- **Optimisation TinyML et SDK.** La quantification et l'élagage des modèles embarqués, ainsi qu'une interface de développement pour des modules tiers, élargiraient l'écosystème.
- **Nouveaux capteurs.** L'activation de la centrale inertielle (détection de chute, stabilisation) et l'intégration optionnelle d'un LiDAR ou d'une caméra événementielle enrichiraient la perception.

9.3.3 Applications potentielles

Au-delà de l'assistance aux personnes malvoyantes, l'architecture modulaire et multi-usage se prête à d'autres domaines évoqués en introduction : la télé-assistance en maintenance industrielle, la formation sur site, et l'exploration en environnement contraint. Le même matériel, reconfiguré par profils, peut servir ces contextes sans refonte logicielle.

9.3.4 *Recherches complémentaires*

- **Validation avec des utilisateurs malvoyants.** C'est la priorité numéro un : une étude avec des participants réels, sous approbation éthique, mesurerait l'impact effectif sur l'autonomie et la qualité de vie, que les métriques techniques ne capturent pas.
- **Étude de robustesse par injection de panne.** La vérification expérimentale de l'isolation des modules (arrêt forcé d'un processus en cours d'exécution) confirmerait formellement la propriété établie ici par conception.
- **Passage à l'échelle.** L'évaluation du broker avec plusieurs casques et de nombreux *topics* validerait des scénarios multi-utilisateurs, par exemple en formation.
- **Comportement en conditions réelles et prolongées.** Des campagnes en environnement extérieur et sur des sessions de plusieurs heures complèteraient la caractérisation menée en laboratoire.

En définitive, Nova ne prétend pas clore le problème de l'assistance embarquée, mais en déplacer la frontière : il montre, mesures à l'appui, qu'une architecture modulaire, hors ligne et reconfigurable est atteignable à faible coût, et il désigne précisément l'obstacle qui reste le calcul de la perception comme la prochaine cible de recherche.

GUIDE INSTALLATION SYSTEME NOVA

A.1 PRÉREQUIS SYSTÈME

Listing A.1 – Installation des dépendances système (Raspberry Pi OS Bookworm)

```

1  # Mise a jour du systeme
2  sudo apt update && sudo apt upgrade -y
3
4  # Broker MQTT Mosquitto
5  sudo apt install -y mosquitto mosquitto-clients
6  sudo systemctl enable mosquitto
7  sudo systemctl start mosquitto
8
9  # Dependances Python et outils systeme
10 sudo apt install -y python3-pip python3-venv git
11 sudo apt install -y tesseract-ocr tesseract-ocr-fra
12 sudo apt install -y espeak-ng libespeak-ng-dev
13 sudo apt install -y libatlas-base-dev libhdf5-dev
14
15 # Bibliothèques Python (via pip)
16 pip install flask flask-socketio paho-mqtt psutil
17 pip install ultralytics onnxruntime
18 pip install opencv-python-headless pytesseract
19 pip install face_recognition vosk picamera2

```

A.2 STRUCTURE DU PROJET

Listing A.2 – Arborescence complète du projet Nova

```

1  nova/
2  |-- nova.py                # Application Flask + orchestrateur
3  |-- utils.py               # ConfigManager (profils JSON)
4  |-- config.json            # Configuration et profils utilisateurs
5  |-- requirements.txt        # Dependances Python
6  |-- templates/             # Interfaces web (Jinja2)
7  |   |-- dashboard.html
8  |   |-- diagnostic.html
9  |   |-- profile_manager.html
10 |   |-- mobile.html
11 |   '-- hud.html
12 |-- plugins/modules/        # Modules fonctionnels independants
13 |   |-- navigation/main.py  # Ultrasons + haptique
14 |   |-- vision_objet/main.py # YOLOv8n + OCR + face_recognition
15 |   |-- assistant_ia/main.py # STT Vosk + TTS eSpeak
16 |   '-- hand_control/main.py # Controle gestuel MediaPipe
17 |-- vision_service.py        # Service Picamera2 (thread isole)
18 '-- modeles/                 # Fichiers modeles IA (.onnx, .tflite)

```

A.3 LANCEMENT DU SYSTÈME

Listing A.3 – Démarrage du système Nova

```

1  # Cloner le depot
2  git clone https://github.com/votre-depot/nova-ai.git
3  cd nova-ai
4
5  # Lancer le broker MQTT (si pas en service systemd)
6  mosquitto -d -p 1883
7
8  # Lancer l'application principale
9  python3 nova.py
10
11 # Accéder à l'interface web depuis n'importe quel navigateur
12 # http://<ip-du-raspberry-pi>:5000

```

A.4 FORMAT DES MESSAGES MQTT

Listing A.4 – Schémas JSON des principaux messages MQTT Nova

```

1  # nova/nav/alert -- Alerte obstacle
2  {
3      "side": "left",          # "left" ou "right"
4      "distance": 85.3,        # en centimetres
5      "level": "HIGH",         # CLEAR / MEDIUM / HIGH
6      "vibration_duty": 100,   # 0-100 (intensite PWM)
7      "tts_message": "Danger, obstacle tres proche",
8      "timestamp": "2026-07-14T09:23:11.456"
9  }
10
11 # nova/vision/detection -- Resultats YOLOv8n
12 {
13     "objects": [
14         {"class": "person", "confidence": 0.87,
15          "bbox": [120, 80, 350, 460]},
16         {"class": "chair", "confidence": 0.72,
17          "bbox": [400, 200, 580, 450]}
18     ],
19     "count": 2,
20     "inference_time_ms": 51.2,
21     "frame_id": 1482
22 }
23
24 # nova/system/status -- Heartbeat module (toutes les 5s)
25 {
26     "module": "vision",
27     "status": "running",
28     "uptime_s": 3621,
29     "fps": 18.4,
30     "cpu_pct": 68.2
31 }

```

BIBLIOGRAPHIE

- [1] Incheol JEONG et al. « YOLOv8-Based XR Smart Glasses Mobility Assistive System for Aiding Outdoor Walking of Visually Impaired Individuals in South Korea ». In : *Electronics* 14.3 (2025), p. 425. DOI : [10.3390/electronics14030425](https://doi.org/10.3390/electronics14030425). URL : <https://www.mdpi.com/2079-9292/14/3/425> (cf. p. [4](#), [6](#), [8](#), [44](#)).
- [2] Issatay TOKMURZIYEV et al. *LLM-Glasses : GenAI-driven Glasses with Haptic Feedback for Navigation of Visually Impaired People*. Présenté à HRI 2026 (ACM/IEEE). 2025. DOI : [10.48550/arXiv.2503.16475](https://doi.org/10.48550/arXiv.2503.16475). arXiv : [2503.16475](https://arxiv.org/abs/2503.16475) [cs.HC]. URL : <https://arxiv.org/abs/2503.16475> (cf. p. [5-8](#), [44](#)).
- [3] N. ANDRIYANOV et S. MIKHAILOVA. « Improving Gesture Recognition Efficiency with MediaPipe and YOLO-Pose ». In : *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences XLVIII-2/W9-2025* (2025), p. 13-18. DOI : [10.5194/isprs-archives-XLVIII-2-W9-2025-13-2025](https://doi.org/10.5194/isprs-archives-XLVIII-2-W9-2025-13-2025). URL : <https://isprs-archives.copernicus.org/articles/XLVIII-2-W9-2025/13/2025/> (cf. p. [5](#)).
- [4] GOOGLE. *MediaPipe Hand Landmarker Task Guide*. Google AI Edge. 2024. URL : https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker (visité le 05/07/2026) (cf. p. [5](#), [12](#)).
- [5] A. ABID, M. JAZIB et M. RIAZ. *RCD-IoT : Enabling Industrial Monitoring and Control with Resource-Constrained Devices Under High Packet Transmission Rates*. 2025. arXiv : [2501.07895](https://arxiv.org/abs/2501.07895) [cs.NI]. URL : <https://arxiv.org/abs/2501.07895> (cf. p. [6](#), [42](#), [46](#)).
- [6] ECLIPSE FOUNDATION. *IoT & Edge Developer Survey Report 2024*. Eclipse Foundation. 2024. URL : <https://iot.eclipse.org/community/resources/iot-surveys/> (visité le 05/07/2026) (cf. p. [6](#)).
- [7] ECLIPSE FOUNDATION. *Eclipse Mosquitto : An Open Source MQTT Broker*. Eclipse Foundation. 2024. URL : <https://mosquitto.org/> (visité le 05/07/2026) (cf. p. [50](#)).
- [8] ORGANISATION MONDIALE DE LA SANTÉ. *Cécité et déficience visuelle*. Organisation mondiale de la Santé. 2023. URL : <https://www.who.int/fr/news-room/fact-sheets/detail/blindness-and-visual-impairment> (visité le 05/07/2026).
- [9] Bharatesh CHAKRAVARTHI, A. A. VERMA et al. *Recent Event Camera Innovations : A Survey*. 2024. arXiv : [2408.13627](https://arxiv.org/abs/2408.13627). URL : <https://arxiv.org/abs/2408.13627>.
- [10] N. A. EZZ-ELDIEN et al. « Empowering Vision Using AI-Enabled Smart Glasses : A Wearable Solution for Enhanced Navigation and Environmental Awareness ». In : *Port Said Engineering Research Journal* 29.4 (2025), p. 172-180.
- [11] B. WANG et al. « A Systematic Literature Review on Integrating AI-Powered Smart Glasses into Digital Health Management for Proactive Healthcare Solutions ». In : *npj Digital Medicine* (2025).
- [12] L. DAL'COL, M. RIEM OLIVEIRA et V. SANTOS. *A Survey of Joint Perception and Prediction for Autonomous Driving*. Manuscrit soumis à IEEE Transactions. 2024.
- [13] P. MAKWANA et al. « AI Enabled Smart Glasses ». In : *International Journal of Innovative Science and Research Technology* 10.3 (2025), p. 869-876.
- [14] D. CAZZATO et F. BONO. « An Application-Driven Survey on Event-Based Neuromorphic Computer Vision ». In : *Information* 15.8 (2024), p. 472. DOI : [10.3390/info15080472](https://doi.org/10.3390/info15080472).
- [15] A. SAXENA et A. SAXENA. *A Multi-Camera Wearable Assistive System for Environmental Awareness in Visually Impaired Users Using Mobile Vision and Real-Time Feedback*. Preprints. 2022.

- [16] Devendra Singh CHAPLOT et al. « Object Goal Navigation using Goal-Oriented Semantic Exploration ». In : *Advances in Neural Information Processing Systems*. 2020.
- [17] H. WANG et al. *Event Camera Meets Mobile Embodied Perception : Abstraction, Algorithm, Acceleration, Application*. Manuscrit soumis à ACM Computing Surveys. 2025.
- [18] S. TENZIN, A. RASSAU et D. CHAI. « Application of Event Cameras and Neuromorphic Computing to VSLAM : A Survey ». In : *Biomimetics* 9.7 (2024), p. 444. DOI : [10.3390/biomimetics9070444](https://doi.org/10.3390/biomimetics9070444).
- [19] H. QIAN et al. « A Review of Multi-Sensor Fusion in Autonomous Driving ». In : *Sensors* 25.19 (2025), p. 6033.
- [20] J. MOOSMANN et al. « Ultra-Efficient On-Device Object Detection on AI-Integrated Smart Glasses With TinyissimoYOLO ». In : *Proceedings of the European Conference on Computer Vision Workshops*. 2024.
- [21] K. PU et al. « ProMemAssist : Exploring Timely Proactive Assistance Through Working Memory Modeling in Multi-Modal Wearable Devices ». In : *Proceedings of the ACM Symposium on User Interface Software and Technology*. UIST 2025. 2025.
- [22] H. WANG et al. « A Survey of the Multi-Sensor Fusion Object Detection Task in Autonomous Driving ». In : *Sensors* 25.9 (2025), p. 2794.
- [23] Joel YE et al. « Auxiliary Tasks and Exploration Enable ObjectGoal Navigation ». In : *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2021.
- [24] C. CIMARELLI et al. « Hardware, Algorithms, and Applications of the Neuromorphic Vision Sensor : A Review ». In : *Sensors* 25.19 (2025), p. 6208.
- [25] G. D'ANGELO et al. « Wandering around : A bioinspired approach to visual attention through object motion sensitivity ». In : *Scientific Reports* (2025).
- [26] Alec RADFORD et al. « Robust Speech Recognition via Large-Scale Weak Supervision ». In : *Proceedings of the 40th International Conference on Machine Learning*. T. 202. Proceedings of Machine Learning Research. PMLR, 2023, p. 28492-28518. URL : <https://proceedings.mlr.press/v202/radford23a.html>.
- [27] OASIS. MQTT Version 5.0 — OASIS Standard. OASIS Open. 2019. URL : <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (visité le 05/07/2026).
- [28] ALPHA CEPHEI. VOSK Offline Speech Recognition API. Alpha Cephei. 2024. URL : <https://alphacephei.com/vosk/> (visité le 05/07/2026).